PostgreSQL

PostgreSQL (/ˈpoʊstgrɛskj uˌɛl/ (i) *POHST-gres-kew-EL*로도 알려진 이 시스템은 확장성과 SQL 준수를 중시하는 무료 오픈 소스 관계형 데이터베이스 관리 시스템(RDBMS)입니다. PostgreSQL은 원자성, 일관성, 격리, 내구성(ACID) 속성, 자동 업데이트 가능한 뷰, 물질화된 뷰, 트리거, 외래 키, 저장 프로시저를 포함한 트랜잭션을 특징으로 합니다. [13] Windows, Linux, macOS, FreeBSD, OpenBSD 등 모든 주요 운영체제에서 지원되며, 단일 머신부터 데이터 웨어하우스, 데이터 레이크[14] 또는 다수의 동시 사용자가 있는 웹 서비스에 이르기까지 다양한 작업을 처리할 수 있습니다.

PostgreSQL 글로벌 개발 그룹은 데이터베이스 엔진과 밀접하게 관련된 구성 요소 개발에만 집중합니다. 이 핵심이 기술적으로 PostgreSQL 자체를 구성하지만, 전통적인 독점 소프트웨어 벤더가 제공할 수 있는 다른 중요한 기능 세트를 제공하는 광범위한 개발자 커뮤니티와 생태계가 존재합니다. 여기에는 지리공간[15] 또는 시간[16] 데이터베이스를 지원하는 특수 목적의 데이터베이스 엔진 기능이나 다른 데이터베이스 제품을 에뮬레이션하는 기능이 포함됩니다. [17][18][19][20] 서드파티에서는 그래픽 사용자 인터페이스와 같은 다양한 사용자 및 기계 인터페이스 기능도 제공됩니다[21][22][23]또는 부하 분산 및 고가용성 툴셋 등이 있습니다. [24] PostgreSQL 개발 그룹의 일부는 아니지만, 사람들, 회사, 제품, 프로젝트로 구성된 대규모 제3자 PostgreSQL 지원 네트워크는 PostgreSQL 데이터베이스 엔진의 채택과 사용에 필수적이며 PostgreSQL 생태계를 구성합니다. [25]

PostgreSQL은 원래 POSTGRES라는 이름으로 불렸으며, 이는 캘리포니아 대학교 버클리에서 개발된 Ingres 데이터베이스의 후속 데이터베이스로 기원을 가리킵니다. [26][27] 1996년, 프로젝트 이름은 SQL 지원을 반영하여 PostgreSQL로 변경되었습니다. 2007년 검토 후, 개발팀은 PostgreSQL이라는 이름과 Postgres라는 별칭을 유지하기로 결정했습니다. [28]

역사

INGRES와 대학교 POSTGRES (1982–1994)

PostgreSQL은 캘리포니아 대학교 버클리의 Ingres 프로젝트에서 발전했습니다. 1982년, Ingres 팀의 리더인 Michael Stonebraker가 Ingres의 독점 버전을 만들기 위해 버클리를 떠났습니다. [26] 1985년 버클리로 돌아와 1980년대 초반 점점 명확해진 현대 데이터베이스 시스템의 문제를 해결하기 위한 앵그르 이후 프로젝트를 시작했습니다. 그는 2014년 이 프로젝트들과 그 안에서 개척된 기법들로 튜링상을 수상했다.

새로운 프로젝트인 POSTGRES는 데이터 타입을 완전히 지원하기 위해 필요한 기능을 최소화하는 것을 목표로 했습니다. [30] 이 기능에는 유형을 정의하고 관계를 완전히 설명하는 기능이 포함되었으며, 이는 널리 사용되지만 전적으로 사용자가 유지할 수 있는 기능이었습니다. POSTGRES에서는 데이터베이스가 관계를 이해하고 규칙을 사용해 관련 테이블에서 정보를 자연스럽게 검색할 수 있었습니다. POSTGRES는 Ingres의 많은 아이디어를 사용했지만, 그 코드는 사용하지 않았습니다. [31]

1986년부터 발표된 논문들이 시스템의 기초를 설명했고, 1988년 ACM SIGMOD 컨퍼런스에서 프로토타입 버전이 공개되었습니다. 팀은 1989년 6월 소수의 사용자에게 버전 1을 공개했고, 1990년 6월에는 규칙 시스템을 재작성한 버전 2를 출시했습니다. 1991년에 출시된 버전 3은 규칙 시스템을 다시 작성하고 다중 스토리지 관리자[32] 지원과 향상된 쿼리 엔진을 추가했습니다. 1993년까지 사용자 수가 지원과 기능 요청으로 프로젝트를 압도하기 시작했습니다. 1994년 6월 30일 버전 4.2[33]를 출시한 후(주로 정리 작업) 프로젝트는 종료되었습니다.

버클리는 MIT 라이선스 변형으로 POSTGRES를 출시하여 다른 개발자들이 어떤 용도로든 코드를 사용할 수 있게 했습니다. 당시 POSTGRES는 Ingres의 영향을 받은 POSTQUEL 쿼리 언어 인터프리터를 사용했으며, 이는 콘솔 애플리케이션과 상호작용적으로 사용할 수 있었습니다.monitor

Postgres95 (1994–1996)

1994년, 버클리 대학원생 Andrew Yu와 Jolly Chen이 POSTQUEL 쿼리 언어 인터프리터를 SQL 쿼리 언어용 인터프리터로 교체하여 Postgres95를 만들었습니다. 콘솔도 .로 대체되었습니다. Yu와 Chen은 1995년 5월 5일 베타 테스터들에게 첫 버전(0.01)을 발표했습니다. Postgres95 버전 1.0은 1995년 9월 5일에 발표되었으며, 소프트웨어가 자유롭게 수정할 수 있도록 보다 자유로운 라이선스가 적용되었습니다.monitorpsql

1996년 7월 8일, Hub.org Networking Services의 Marc Fournier가 오픈 소스 개발 노력을 위한 최초의 비대학 개발 서버를 제공했습니다. [3] Bruce Momjian과 Vadim B. Mikheev의 참여로 버클리에서 물려받은 코드를 안정화하는 작업이 시작되었습니다.

PostgreSQL (1996–현재)

1996년, 프로젝트 명칭이 SQL 지원을 반영하여 PostgreSQL로 변경되었습니다. 웹사이트 PostgreSQL.org 온라인 존재감은 1996년 10월 22일에 시작되었습니다. [34] 첫 번째 PostgreSQL 릴리스는 1997년 1월 29일에 버전 6.0을 형성했습니다. 그 이후로 전 세계 개발자와 자원봉사자들이 The PostgreSQL 글로벌 개발 그룹이라는 이름으로 소프트웨어를 유지보수해 왔습니다. [2]

프로젝트는 무료 오픈 소스 소프트웨어인 PostgreSQL 라이선스 하에 계속 릴리스를 제공하고 있습니다. 코드는 독점 벤더, 지원 회사, 오픈 소스 프로그래머들의 기여에서 나옵니다.

2025년 기준으로 PostgreSQL은 비동기 I/O(AIO) 구현이 주목받는 주요 릴리스 18에 있으며, 데이터베이스 사용자가 리드어헤드와 순차 스캔과 같은 동시 I/O 작업을 수행할 수 있게 합니다.[35]

다중 버전 동시성 제어(MVCC)

PostgreSQL은 다중 버전 동시성 제어(MVCC)를 통해 동시성을 관리하며, 이는 각 트랜잭션에 데이터베이스의 "스냅샷"을 제공하여 다른 트랜잭션에 영향을 주지 않고 변경을 할 수 있게 합니다. 이로 인해 읽기 잠금이 거의 필요 없어지고, 데이터베이스가 ACID 원칙을 유지하도록 보장합니다. PostgreSQL은 Read Uncommitted, Read Committed, Repeatable Read, 직렬화 가능한 네 가지 트랜잭션 격리를 제공합니다. PostgreSQL은 더티 리드에 면역이기 때문에, Read Uncommitted(컨밋드 없음) 트랜잭션 격리 레벨을 요청하면 Read 커밋드가 제공됩니다. PostgreSQL은 직렬화 가능한 스냅샷 격리(SSI) 방식을 통해 완전한 직렬화 가능성을 지원합니다. [36] PostgreSQL MVCC 구현은 기존 행을 업데이트하는 무거운 쓰기 부하 시 성능 문제를 겪기 쉽고, 이를 조정해야 합니다. [37]

저장 및 복제

복제

PostgreSQL은 변경 사항을 비동기적으로 복제 노드에 전송하는 기반 내장 이진 복제(write-ahead logs, WAL)를 포함하며, 복제된 노드에 대해 읽기 전용 쿼리를 실행할 수 있습니다. 이를 통해 읽기 트래픽을 여러 노드에 효율적으로 분할할 수 있습니다. 유사한 읽기 확장을 가능하게 한 초기 복제 소프트웨어들은 보통 마스터에 복제 트리거를 추가하는 방식으로 부하를 증가시켰습니다.

PostgreSQL은 내장된 동기식 복제[38]를 포함하고 있어, 각 쓰기 트랜잭션마다 마스터가 적어도 하나의 복제 노드가 트랜잭션 로그에 데이터를 기록할 때까지 기다립니다. 다른 데이터베이스 시스템과 달리, 트랜잭션의 내구도(비동기 또는 동기 관계)는 데이터베이스별, 사용자별, 세션별, 심지어 트랜잭션별로도 지정할 수 있습니다. 이는 이러한 보장이 필요하지 않은 워크로드에 유용하며, 트랜잭션이 동기 대기(Synchronous standby)에 도달했는지 확인해야 하므로 성능 저하를 초래하기 때문에 모든 데이터에 적용되는 것은 아닐 수 있습니다.

대기 서버는 동기식 또는 비동기식일 수 있습니다. 동기식 대기 서버는 동기식 복제 후보가 될 서버를 결정하는 구성에서 지정할 수 있습니다. 목록에서 가장 먼저 활동 중인 서버가 현재 동기식 서버로 사용될 예정입니다. 이 과정이 실패하면 시스템은 다음 라인으로 넘어갑니다.

PostgreSQL의 복제는 특히 튜플이 대기 서버에서 조기 삭제될 때 주 서버와 대기 서버 간 충돌을 유발할 수 있으며, 쿼리가 계속 접근하고 있음에도 불구하고 그렇습니다. [39] 이를 해결하기 위해 PostgreSQL은 피드백 플래그[40]를 포함하여, 대기 서버가 진행 중인 쿼리를 주 서버에 미리 알리도록 하여 이러한 충돌을 완화합니다.

동기식 다중 마스터 복제는 PostgreSQL 코어에 포함되어 있지 않습니다. PostgreSQL을 기반으로 한 Postgres-XC는 확장 가능한 동기식 다중 마스터 복제를 제공합니다. [41] PostgreSQL과 동일한 라이선스 하에 라이선스가 적용되어 있습니다. 관련 프로젝트로는 Postgres-XL이라는 이름이 있습니다. Postgres-R도 또 다른 포크입니다. [42] 양방향 복제(BDR)는 PostgreSQL을 위한 비동기식 다중 마스터 복제 시스템입니다. [43]

repmgr와 같은 도구는 복제 클러스터 관리를 더 쉽게 만듭니다.

여러 비동기 트리거 기반 복제 패키지가 제공됩니다. 이들은 확장된 핵심 능력이 도입된 후에도 여전히 유용하며, 전체 데이터베이스 클러스터의 이진 복제가 부적절한 상황에서 사용할 수 있습니다:

- 슬로니-I
- Londiste, SkyTools 산하 (Skype 개발)
- Bucardo 멀티마스터 복제(Backcountry.com 개발)[44]
- SymmetricDS 멀티마스터, 멀티티어 복제

색인

PostgreSQL은 일반 B-트리 및 해시 테이블 인덱스에 대한 내장 지원과 네 가지 인덱스 접근 방식을 포함합니다: 일반화된 탐색 트리(GiST), 일반화된 역인덱스(GIN), 공간 분할 GiST(SP-GiST)[45][45], 블록 범위 인덱스(BRIN). 또한 사용자 정의 인덱스 메서드를 만들 수 있지만, 이는 상당히 복잡한 과정입니다. PostgreSQL의 인덱스는 다음과 같은 기능도 지원합니다:

- 식의 인덱스는 단순히 열의 값이 아니라 식이나 함수의 결과의 인덱스로 만들 수 있습니다.
- 부분 인덱스는 테이블의 일부만 인덱싱하는 것으로, CREATE INDEX 문장 끝에 WHERE 절을 추가하여 생성할 수 있습니다. 이로 인해 더 작은 인덱스를 만들 수 있습니다.
- 플래너는 복잡한 쿼리를 위해 여러 인덱스를 함께 사용할 수 있으며, 임시 메모리 내 비트맵 인덱스 연산을 사용할 수 있습니다(큰 팩트 테이블을 스타 스키마와 같은 작은 차원 테이블과 연결하는 데 유용한 데이터 웨어하우스 응용 분야에 유용합니다).
- *k*-최근접 이웃(*k-NN*) 인덱싱(KNN-GiST[46]라고도 함)은 지정된 값에 대한 "가장 가까운 값"을 효율적으로 검색할 수 있게 하며, 유사한 단어나 지리공간 데이터가 있는 가까운 객체 또는 위치를 찾는 데 유용합니다. 이는 값의 철저한 일치 없이도 달성됩니다.
- 인덱스 전용 스캔은 시스템이 메인 테이블에 접근하지 않고도 인덱스에서 데이터를 가져올 수 있게 해줍니다.

- 블록 범위 지수(BRIN).

스키마

PostgreSQL 스키마는 같은 종류와 이름의 객체들이 단일 데이터베이스 내에서 공존할 수 있도록 하는 네임스페이스입니다. 이들은 각 테이블의 데이터가 다른 테이블 내 데이터와 어떻게 연결되는지 정의하는 추상적이고 구조적이며 조직적인 명세인 데이터베이스 스키마와 혼동해서는 안 됩니다. 역할이나 테이블스페이스와 같은 일부 전역 객체를 제외한 모든 PostgreSQL 데이터베이스 객체는 스키마 내에 존재합니다. 중첩될 수 없으며, 스키마는 스키마를 포함할 수 없습니다. 권한 시스템은 스키마와 그 내용에 대한 접근을 제어합니다. 기본적으로 새로 생성된 데이터베이스는 퍼블릭이라는 단일 스키마만 있지만, 다른 스키마를 추가할 수 있고 퍼블릭 스키마가 필수는 아닙니다.

설정은 PostgreSQL이 사전 접두사가 없는 자격 없는 객체에 대해 스키마를 검사하는 순서를 결정합니다. 기본적으로 (현재 연결된 데이터베이스 사용자를 의미합니다)로 설정되어 있습니다. 이 기본값은 데이터베이스나 역할 수준에서 설정할 수 있지만, 세션 매개변수이기 때문에 클라이언트 세션 중에도 자유롭게(심지어 여러 번) 변경할 수 있으며, 이는 해당 세션에만 영향을 미칩니다. `search_path$user, public$user`

존재하지 않는 스키마나 로그인한 사용자가 접근할 수 없는 기타 스키마는 객체 조회 시 조용히 건너뛸 수 search_path.

새로운 객체는 search_path에서 가장 먼저 나타나는 유효한 스키마(접근 가능한 스키마)에서 생성됩니다.

데이터 타입

다음과 같은 다양한 네이티브 데이터 타입이 지원됩니다:

- Boolean
- Arbitrary-precision numerics
- Character (text, varchar, char)
- Binary
- Date/time (timestamp/time with/without time zone, date, interval)
- Money
- Enum
- Bit strings
- Text search type
- Composite
- HStore, an extension enabled key–value store within PostgreSQL[47]
- Arrays (variable-length and can be of any data type, including text and composite types) up to 1 GB in total storage size
- Geometric primitives
- IPv4 and IPv6 addresses
- Classless Inter-Domain Routing (CIDR) blocks and MAC addresses
- XML supporting XPath queries
- Universally unique identifier (UUID)
- JavaScript Object Notation (JSON), and a faster binary JSONB (not the same as BSON[48])

또한 사용자는 자신만의 데이터 타입을 만들 수 있으며, 이는 일반적으로 PostgreSQL의 인덱싱 인프라인 GiST, GIN, SP-GiST를 통해 완전히 인덱싱 가능하게 만들 수 있습니다. 예를 들어, PostgreSQL 프로젝트의 지리 정보 시스템(GIS) 데이터 타입이 있습니다.

도메인이라는 데이터 타입도 있는데, 이는 다른 데이터 타입과 동일하지만 해당 도메인 생성자가 정의하는 선택적 제약 조건을 가집니다. 즉, 도메인을 사용하는 열에 입력되는 모든 데이터는 도메인의 일부로 정의된 제약 조건을 따라야 합니다.

범위 데이터를 나타내는 데이터 타입을 사용할 수 있는데, 이를 범위 타입이라고 합니다. 이들은 이산 범위(예: 모든 정수 값 1부터 10까지)나 연속적인 범위(예: 오전 10시부터 11시 사이의 시간대)일 수 있습니다. 내장된 범위 유형에는 정수, 큰 정수, 소수점, 시간대가 있는 또는 없는 시간표, 날짜 등이 포함됩니다.

사용자 지정 범위 유형을 생성하여 새로운 유형의 범위를 사용할 수 있는데, 예를 들어 IP 주소 범위는 inet 타입을 기준으로 사용하거나, float 데이터 타입을 기준으로 사용하는 float 범위 등이 있습니다. 범위 유형은 각각 와 를 사용하여 포괄적 범위 경계와 배타적 범위를 지원합니다. (예: 는 4부터 9까지의 모든 정수를 나타낸다.) 범위 유형은 겹침, 격리, 권리 등을 확인하는 기존 운영자와도 호환됩니다. `[]()[4,9)`

사용자 정의 객체

데이터베이스 내 거의 모든 객체의 새로운 유형을 생성할 수 있으며, 여기에는 다음과 같은 것이 포함됩니다:

- Casts
- Conversions
- Data types
- Data domains
- Functions, including aggregate functions and window functions
- Indexes including custom indexes for custom types
- Operators (existing ones can be overloaded)
- Procedural languages

상속

테이블은 부모 테이블로부터 특성을 상속하도록 설정할 수 있습니다. 자식 테이블의 데이터는 부모 테이블에 존재하는 것처럼 보이지만, 부모 테이블에서 ONLY 키워드를 사용해 데이터를 선택하지 않는 한 부모 테이블에 열을 추가하면 그 열이 자식 테이블에 나타납니다. `SELECT * FROM ONLY parent_table;`

상속은 테이블 분할을 구현하는 데 사용될 수 있으며, 트리거나 규칙을 사용하여 적절한 자식 테이블에 부모 테이블로 삽입을 지시합니다.

이 기능은 완전히 지원되지 않습니다. 특히, 테이블 제약 조건은 현재 상속되지 않습니다. 부모 테이블의 모든 체크 제약 조건과 null이 아닌 조건은 자식들에게 자동으로 상속됩니다. 고유 키, 주 키 제약, 외래 키 제약 등 다른 유형의 제약은 상속되지 않습니다.

상속은 엔터티-관계 다이어그램(ERD)에 표현된 일반화 계층의 특징을 PostgreSQL 데이터베이스에 직접 매핑하는 방법을 제공합니다.

기타 저장 기능

- 외래키 제약, 열 제약, 행 검사를 포함한 참조 무결성 제약
- 이진 및 텍스트 대형 객체 저장
- 테이블스페이스

- 열별 연합
- 온라인 백업
- 기록 기록을 사용하여 구현된 시점 복구
- pg_upgrade 있는 현장 업그레이드로 다운타임을 줄이세요

제어 및 연결성

외부 데이터 래퍼

PostgreSQL은 외부 데이터 래퍼(FDW)를 통해 데이터를 가져오기 위해 다른 시스템과 연결할 수 있습니다. [49] 이들은 파일 시스템, 다른 관계형 데이터베이스 관리 시스템(RDBMS), 웹 서비스 등 어떤 데이터 소스의 형태를 취할 수 있습니다. 즉, 일반적인 데이터베이스 쿼리는 이러한 데이터 소스를 일반 테이블처럼 사용할 수 있으며, 여러 데이터 소스를 결합할 수도 있습니다.

인터페이스

PostgreSQL은 애플리케이션이 데이터베이스 서버에 연결할 수 있도록 하는 이진 통신 프로토콜을 지원합니다. 이 프로토콜은 버전이 지정되어 있으며(현재 3.0, PostgreSQL 7.4 기준), 상세한 사양이 있습니다. [50]

이 통신 프로토콜의 공식 클라이언트 구현은 C API libpq입니다. [51] 또한, 공식 지원되는 ECPG 도구는 SQL 명령을 C 코드에 삽입할 수 있도록 합니다. [52] 두 가지 모두 표준 PostgreSQL 배포판의 일부입니다. [53]

PostgreSQL에 연결하기 위한 서드파티 라이브러리는 C++, [54] Java,[55] Julia,[56][57][58] Python,[59] Node.js,[60] Go,[61], Rust 등 다양한 프로그래밍 언어에서 제공됩니다. [62]

절차적 언어

절차적 언어는 개발자가 데이터베이스를 확장하여 사용자 정의 서브루틴(함수), 즉 저장 프로시저라고 부릅니다. 이 함수들은 데이터베이스 트리거(특정 데이터 수정 시 호출되는 함수)와 사용자 지정 데이터 유형 및 집계 함수를 구축하는 데 사용할 수 있습니다. [63] 절차적 언어는 함수를 정의하지 않고도 SQL 수준에서 DO 명령어를 사용하여 호출할 수 있습니다. [64]

언어는 두 그룹으로 나뉩니다: 안전한 언어로 작성된 프로시저는 샌드박스되어 있어 모든 사용자가 안전하게 생성하고 사용할 수 있습니다. 안전하지 않은 언어로 작성된 프로시저는 데이터베이스의 보안 제한을 우회할 수 있기 때문에 슈퍼사용자만 생성할 수 있으며, 데이터베이스 외부의 소스에도 접근할 수 있습니다. Perl 같은 일부 언어는 안전한 버전과 안전하지 않은 버전을 모두 제공합니다.

PostgreSQL은 세 가지 절차적 언어에 대한 내장된 지원을 제공합니다:

- 기본 SQL (안전). 더 단순한 SQL 함수는 호출(SQL) 쿼리에 인라인으로 확장할 수 있어, 함수 호출 오버헤드를 줄이고 쿼리 옵티마이저가 함수 내부를 '볼 수 있게' 됩니다.
- 절차적 언어/PostgreSQL (PL/pgSQL)(안전)은 Oracle의 절차적 언어 for SQL(PL/SQL) 절차적 언어와 SQL/Persistent Stored Module(SQL/PSM)과 유사합니다.
- C(안전하지 않음)는 하나 이상의 사용자 지정 공유 라이브러리를 데이터베이스에 불러올 수 있게 해줍니다. C로 작성된 함수는 최고의 성능을 제공하지만, 코드의 버그가 발생해 데이터베이스가 손상될 수 있습니다. 대부분의 내장 함수는 C로 작성되어 있습니다.

또한, PostgreSQL은 확장 기능을 통해 절차적 언어를 데이터베이스에 로드할 수 있습니다. PostgreSQL에는 기본 Perl, Tcl, Python 지원을 위한 세 가지 언어 확장 기능이 포함되어 있습니다(PL/Python PL/Perl PL/Tcl). 파이썬은 현재 Python 3를 사용하며, PostgreSQL 15부터 중단된 Python 2는 더 이상 지원되지 않습니다. 두 프로그램 모두 이전에는 기본적으로 Python 2를 지원했으며, 구버전과 신버전 모두 같은 세션에서 사용할 수 없었습니다. [65] 외부 프로젝트들은 PL/Java, JavaScript(PL/V8), PL/Julia,[58] PL/R,[67] PL/Ruby 등 다양한 언어를 지원합니다.

트리거

트리거는 SQL 데이터 조작 언어(DML) 문장의 동작에 의해 트리거되는 이벤트입니다. 예를 들어, INSERT 문은 문장 값이 유효한지 확인하는 트리거를 활성화할 수 있습니다. 대부분의 트리거는 INSERT 또는 UPDATE 문으로만 활성화됩니다.

트리거는 완전히 지원되며 테이블에 부착할 수 있습니다. 트리거는 컬럼 단위와 조건부 모두가 될 수 있는데, UPDATE 트리거는 테이블의 특정 열을 대상으로 할 수 있고, 트리거의 WHERE 절에서 지정한 조건 집합 하에서 실행하도록 명령할 수 있습니다. 트리거는 INSTEAD OF 조건을 사용하여 뷰에 부착할 수 있습니다. 여러 트리거가 알파벳 순서대로 발사됩니다. PL/pgSQL로 작성된 네이티브 함수를 호출하는 것 외에도, 트리거는 PL/Python이나 PL/Perl 같은 다른 언어로 작성된 함수도 호출할 수 있습니다.

비동기 알림

PostgreSQL은 NOTIFY, LISTEN 및 UNLISTEN 명령을 통해 접근하는 비동기식 메시징 시스템을 제공합니다. 세션은 사용자가 지정한 채널과 선택적 페이로드와 함께 NOTIFY 명령을 발송하여 특정 이벤트 발생을 표시할 수 있습니다. 다른 세션들은 LISTEN 명령을 실행하여 특정 채널을 들을 수 있도록 이러한 이벤트를 감지할 수 있습니다. 이 기능은 테이블이 업데이트되었을 때 다른 세션에 알리거나, 특정 동작이 수행되었는지 감지하는 별도의 애플리케이션에 활용될 수 있습니다. 이러한 시스템은 애플리케이션이 아직 변경된 점이 있는지 확인하기 위해 지속적으로 폴링할 필요를 없애고 불필요한 오버헤드를 줄여줍니다. 알림은 완전히 트랜잭션 형식으로, 메시지가 전송된 트랜잭션이 커밋될 때까지 전송되지 않습니다. 이로 인해 수행 중인 동작에 대해 메시지를 보내고 그 후 롤백되는 문제가 해결됩니다.

많은 PostgreSQL용 커넥터(libpq, JDBC, Npgsql, psycopg, node.js 포함)가 이 알림 시스템을 지원하여 외부 애플리케이션에서도 사용할 수 있습니다.

PostgreSQL은 LISTEN과 FOR UPDATE SKIP LOCK을 결합하여 효과적이고 지속적인 "PUB/SUB" 서버 또는 작업 서버 역할을 할 수 있습니다. [68][69][70]

규칙

규칙은 들어오는 쿼리의 "쿼리 트리"를 다시 작성할 수 있게 합니다; 이들은 SQL을 위한 자동으로 호출되는 매크로 언어입니다. "쿼리 재작성 규칙"은 테이블/클래스에 부착되며, 들어오는 DML(선택, 삽입, 업데이트, 삭제)을 하나 이상의 쿼리로 "재작성"하여 원래 DML 문장을 대체하거나 추가로 실행합니다. 쿼리 재작성은 DML 문장 파싱 이후, 쿼리 계획 전에 이루어집니다.

규칙이 제공하는 기능은 거의 모든 면에서 새로운 유형의 트리거가 도입되면서 복제되었습니다. 동등한 규칙을 사용할 때보다 트리거 행동과 상호작용을 추론하기 쉽기 때문에 규칙보다 트리거 사용이 보통 선호됩니다.

기타 쿼리 기능

- Transactions
- Full-text search
- Views

    - Materialized views[71]
    - Updateable views[72]
    - Recursive views[73]
- Inner, outer (full, left, and right), and cross joins
- Sub-selects
    - Correlated sub-queries[74]
- Regular expressions[75]
- 공통 테이블 표현식과 작성 가능한 공통 테이블 표현식
- 전송 계층 보안(TLS)을 통한 암호화된 연결; 현재 버전은 해당 설정 옵션을 사용해도 취약한 SSL을 사용하지 않습니다[76]
- Domains
- Savepoints
- Two-phase commit
- The Oversized-Attribute Storage Technique (TOAST)은 큰 테이블 속성(예: 큰 MIME 첨부파일이나 XML 메시지)을 별도의 영역에 투명하게 저장하는 데 사용되며, 자동 압축이 적용됩니다.
- 임베디드 SQL은 전처리기를 사용하여 구현됩니다. SQL 코드는 먼저 C 코드에 내장되어 작성됩니다. 그 후 코드를 ECPG 전처리기를 통해 실행되는데, ECPG는 SQL을 코드 라이브러리 호출로 대체합니다. 그 후 코드를 C 컴파일러로 컴파일할 수 있습니다. 임베딩은 C++에서도 작동하지만 모든 C++ 구문을 인식하지는 않습니다.

동시성 모델

PostgreSQL 서버는 프로세스 기반(스레드 기반이 아님)이며, 각 데이터베이스 세션당 하나의 운영 체제 프로세스를 사용합니다. 운영체제는 여러 세션을 모든 사용 가능한 CPU에 자동으로 분산시킵니다. 여러 종류의 쿼리는 여러 백그라운드 워커 프로세스에 병렬화되어 여러 CPU나 코어를 활용할 수 있습니다. [77] 클라이언트 애플리케이션은 스레드를 사용하고 각 스레드로부터 여러 데이터베이스 연결을 생성할 수 있습니다. [78]

보안

PostgreSQL은 역할별로 내부 보안을 관리합니다. 역할은 일반적으로 사용자(로그인 가능한 역할) 또는 그룹(다른 역할이 구성원인 역할)으로 간주됩니다. 권한은 열까지 어떤 객체에 대해서도 부여 또는 취소할 수 있으며, 데이터베이스, 스키마, 테이블, 행 수준에서 객체의 가시성/생성/변경/삭제를 허용하거나 금지할 수 있습니다.

PostgreSQL의 SECURITY LABEL 기능(SQL 표준의 확장)은 추가적인 보안을 가능하게 하며; 보안 강화 리눅스(SELinux) 보안 정책을 기반으로 한 라벨 기반 필수 접근 제어(MAC)를 지원하는 번들 로딩 모듈과 함께 제공됩니다. [79][80]

PostgreSQL은 다음과 같은 다양한 외부 인증 메커니즘을 기본적으로 지원합니다:

- Password: either SCRAM-SHA-256,[81] MD5 or plain-text
- Generic Security Services Application Program Interface (GSSAPI)
- Security Support Provider Interface (SSPI)
- Kerberos
- ident (maps O/S user-name as provided by an ident server to database user-name)

- Peer (maps local user name to database user name)
- Lightweight Directory Access Protocol (LDAP)
    - Active Directory (AD)
- RADIUS
- Certificate
- Pluggable authentication module (PAM)

The GSSAPI, SSPI, Kerberos, peer, ident and certificate methods can also use a specified "map" file that lists which users matched by that authentication system are allowed to connect as a specific database user.

These methods are specified in the cluster's host-based authentication configuration file (), which determines what connections are allowed. This allows control over which user can connect to which database, where they can connect from (IP address, IP address range, domain socket), which authentication system will be enforced, and whether the connection must use Transport Layer Security (TLS). `pg_hba.conf`

표준 준수

PostgreSQL은 최신 SQL 표준에 대한 높은 준수를 주장하지만 완전하지는 않습니다("2024년 9월 버전 17 릴리스 기준으로 PostgreSQL은 SQL:2023 Core 적합성에 필수인 177개 기능 중 최소 170개를 준수하며, 다른 데이터베이스는 완전히 준수하지 않습니다[82]). 예외 중 하나는 테이블 이름이나 열명과 같은 인용번호가 없는 식별자 처리입니다. PostgreSQL에서는 내부적으로 소문자로 접혀 있지만[83] 표준에서는 인용번호가 없는 식별자를 대문자로 접어야 한다고 명시되어 있습니다. 따라서 는 표준에 따르면 '그렇지 않다'와 동등해야 합니다. 다른 단점으로는 트랜잭션 중 행 버전을 자동으로 기록할 수 있는 시간 데이블이 없다는 점(FOR SYSTEM TIME PREDICATE)이 있지만, 비교적 SQL 호환 서드파티 확장 기능도 있습니다. [16]`FooFOOfoo`

벤치마크 및 성능

PostgreSQL에 대한 비공식 성능 연구가 많이 수행되었습니다. [84] 확장 가능성을 높이기 위한 성능 개선은 8.1 버전부터 크게 시작되었습니다. 버전 8.0과 버전 8.4 간의 간단한 벤치마크 결과, 후자는 읽기 전용 작업에서 10배 이상 빠르고, 읽기와 쓰기 작업 모두에서 최소 7.5배 이상 빠른 것으로 나타났습니다. [85]

최초의 업계 표준이자 동료 검증을 거친 벤치마크는 2007년 6월에 완료되었으며, Sun Java System Application Server(GlassFish의 독점 버전) 9.0 플랫폼 에디션, UltraSPARC T1 기반 Sun Fire 서버, PostgreSQL 8.2를 사용했습니다. [86] 이 778.14 SPECjAppServer2004 JOPS@Standard 결과는 Itanium 기반 HP-UX 시스템에서 Oracle 10을 탑재한 874 JOPS@Standard과 비교해 우수합니다. [84]

2007년 8월, Sun은 향상된 벤치마크 점수인 813.73 SPECjAppServer2004 JOPS@Standard을 제출했습니다. 시험 중인 시스템이 할인된 가격에 따라 가격 대비 성능은 $84.98/JOPS에서 $70.57/JOPS로 향상되었습니다. [87]

PostgreSQL의 기본 구성은 데이터베이스 블록 캐싱 및 정렬과 같은 성능에 중요한 목적을 위해 전용 메모리를 소량만 사용합니다. 이 제한은 주로 이전 운영체제가 큰 공유 메모리 블록을 할당하기 위해 커널 변경을 요구했기 때문입니다. [88] PostgreSQL.org 위키에서 기본적인 권장 공연 연습에 대한 조언을 제공합니다. [89]

2012년 4월, EnterpriseDB의 Robert Haas는 64코어 서버를 사용해 PostgreSQL 9.2의 선형 CPU 확장성을 시연했습니다. [90]

Matloob Khushi는 PostgreSQL 9.0과 MySQL 5.6.15 간의 유전체 데이터 처리 능력에 대한 벤치마킹을 수행했습니다. 그의 성능 분석에서 그는 PostgreSQL이 8만 개의 무작위 인간 DNA 영역을 형성하는 두 개의 데이터셋을 사용해 MySQL보다 8배 빠르게 겹치는 유전체 영역을 추출한다는 사실을 발견했습니다. PostgreSQL의 삽입 및 데이터 업로드도 더 우수했으나, 두 데이터베이스의 일반적인 검색 능력은 거의 비슷했습니다. [91]

플랫폼

PostgreSQL은 다음 운영체제에서 사용할 수 있습니다: Linux(모든 최근 배포판), macOS 버전 10.14 및 그 이후에서 사용 및 테스트된 64비트 ARM 및 x86-64 설치 프로그램, [92] Windows(64비트 Windows Server 2022 및 2016[93]에서 설치 프로그램 사용 및 테스트됨), FreeBSD, OpenBSD,[94] NetBSD, DragonFlyBSD, 그리고 공식적이지만(비공식적이지만 아마도 제공 가능한) 이진 실행 파일 Solaris,[95]와 illumos 등이 있습니다.

PostgreSQL은 다음 명령어 집합 아키텍처(및 운영체제) 중 어느 것에서든 작동할 것으로 기대됩니다: Windows 및 기타 운영체제의 64비트 x86-64 및 32비트 x86; 이들은 윈도우 외에도 64비트 ARM[96]과 구형 32비트 ARM(라즈베리 파이의 ARMv6 포함), RISC-V, z/아키텍처, S/390, PowerPC(64비트 Power ISA 포함), SPARC(역시 64비트), MIPS, PA-RISC 등에서 지원됩니다. 또한 몇 년간 테스트되지 않았지만 최신 버전에서는 다른 플랫폼에서도 작동하는 것으로 알려져 있습니다. [98]

PSQL

PostgreSQL의 주요 프론트엔드는 명령줄 프로그램으로, SQL 쿼리를 직접 입력하거나 파일에서 실행할 수 있습니다. 또한 psql은 스크립트 작성과 다양한 작업 자동화를 용이하게 하는 여러 메타 명령어와 다양한 셸 유사 기능을 제공합니다; 예를 들어, 객체 이름의 탭 완성과 SQL 문법이 있습니다.`psql`

pgAdmin

pgAdmin 패키지는 PostgreSQL용 무료 오픈 소스 그래픽 사용자 인터페이스(GUI) 관리 도구로, 여러 컴퓨터 플랫폼에서 지원됩니다. [100] 이 프로그램은 12개 이상의 언어로 제공됩니다. 첫 번째 프로토타입인 pgManager는 1998년부터 PostgreSQL 6.3.2용으로 작성되었으며, 이후 몇 달에 GNU 일반 공중 사용 허락서(GPL) 하에 pgAdmin으로 재작성되어 공개되었습니다. 두 번째 버전(pgAdmin II라는 이름)은 완전한 재작성으로, 2002년 1월 16일에 처음 공개되었습니다. 세 번째 버전인 pgAdmin III는 원래 아티스틱 라이선스(Artistic License)로 출시되었고, 이후 PostgreSQL과 동일한 라이선스로 출시되었습니다. Visual Basic으로 작성된 이전 버전과 달리, pgAdmin III는 c++로 작성되었으며, wxWidgets[101] 프레임워크를 사용하여 대부분의 일반적인 운영체제에서 실행할 수 있습니다. 쿼리 도구에는 pgScript라는 스크립팅 언어가 포함되어 있어 관리자 및 개발 작업을 지원합니다. 2014년 12월, pgAdmin 프로젝트 창립자이자 주요 개발자인 Dave Page[102]는 웹 기반 모델로의 전환과 함께 클라우드 배포를 용이하게 하기 위한 pgAdmin 4 작업이 시작되었다고 발표했습니다. [103] 2016년에 pgAdmin 4가 출시되었습니다. pgAdmin 4 백엔드는 Python으로 작성되었으며, Flask와 Qt 프레임워크를 사용했습니다. [104]

phpPgAdmin

phpPgAdmin은 PHP로 작성된 웹 기반 PostgreSQL용 관리 도구로, 원래 MySQL 관리를 위해 작성된 인기 있는 phpMyAdmin 인터페이스를 기반으로 합니다. [105]

PostgreSQL Studio

PostgreSQL Studio는 사용자가 웹 기반 콘솔에서 필수적인 PostgreSQL 데이터베이스 개발 작업을 수행할 수 있도록 합니다.

PostgreSQL Studio는 사용자가 방화벽을 열지 않고도 클라우드 데이터베이스를 다룰 수 있게 해줍니다. [106]

TeamPostgreSQL

PostgreSQL용 AJAX/자바스크립트 기반 웹 인터페이스. 웹 브라우저를 통해 데이터 및 데이터베이스 객체를 탐색, 유지, 생성할 수 있습니다. 인터페이스는 자동 완성 기능이 있는 탭형 SQL 편집기, 행 편집 위젯, 행과 테이블 간 외래키 탐색, 자주 사용되는 스크립트에 대한 즐겨찾기 관리 등 다양한 기능을 제공합니다. 웹 인터페이스와 데이터베이스 연결 모두에 대해 SSH를 지원합니다. Windows, Macintosh, Linux용 설치 프로그램이 제공되며, 스크립트에서 실행되는 간단한 크로스 플랫폼 아카이브도 있습니다. [107]

Libreoffice,, OpenOffice.org

LibreOffice와 OpenOffice.org Base는 PostgreSQL의 프론트엔드로 사용할 수 있습니다. [108][109]

pgBadger

pgBadger PostgreSQL 로그 분석기는 PostgreSQL 로그 파일에서 상세 보고서를 생성합니다. [110]

pgDevOps

pgDevOps는 여러 PostgreSQL 버전, 확장 기능, 커뮤니티 컴포넌트를 설치하고 관리하고, SQL 쿼리를 개발하며, 실행 중인 데이터베이스를 모니터링하고 성능 문제를 발견할 수 있는 웹 도구 모음입니다. [111]

Adminer

Adminer는 PHP로 작성된 PostgreSQL 및 기타 용종을 위한 간단한 웹 기반 관리 도구입니다.

pgBackRest

pgBackRest는 PostgreSQL용 백업 및 복원 도구로, 완전 백업, 차별, 증분 백업을 지원합니다. [112]

PGAUDIT

pgaudit는 PostgreSQL의 확장 기능으로, PostgreSQL에서 제공하는 표준 로깅 기능을 통해 상세한 세션 및/또는 객체 감사 로깅을 제공합니다. [113]

WAL-E

WAL-E는 Python으로 작성된 물리적(WAL 기반) 백업을 지원하는 PostgreSQL용 백업 및 복원 도구입니다. [114]

DBeever

DBeaver는 PostgreSQL용 무료 오픈 소스 GUI 관리 도구로, Visual Entity Diagrams와 Intellisense 기능을 갖추고 있습니다. 또한 상업용 PRO 라이선스도 보유하고 있습니다.

Postgresus

Postgresus는 외부 소스(S3, NAS, FTP, Google Drive, Google Cloud 등)와 외부 서비스(Slack, Discord, Telegram, SMTP 등)에 대한 알림을 지원하는 GUI가 포함된 오픈 소스 백업 도구입니다. [115]

여러 회사가 PostgreSQL용 독점 도구를 제공합니다. 이들은 종종 다양한 특정 데이터베이스 제품에 맞게 조정된 범용 코어로 구성됩니다. 이 도구들은 주로 오픈 소스 도구와 관리 기능을 공유하지만, 데이터 모델링, 가져오기, 내보내기 또는 보고에 대한 개선을 제공합니다.

주요 사용자

PostgreSQL을 주요 데이터베이스로 사용하는 주요 조직과 제품은 다음과 같습니다:

- 마이크로소프트는 8억 대가 넘는 윈도우 기기에서 2만 가지 유형의 지표를 분석하며 윈도우 업데이트 품질을 추적하는 페타바이트 규모의 "릴리스 품질 뷰"(RQV) 분석 대시보드에 사용되었습니다. [116]
- 2009년, 소셜 네트워킹 웹사이트 Myspace는 수정되지 않은 PostgreSQL 기반으로 구축된 Aster Data Systems의 nCluster 데이터베이스를 데이터 웨어하우징에 사용했습니다. [117][118]
- Geni.com 주요 유전자 계보 데이터베이스로 PostgreSQL을 사용합니다. [119]
- OpenStreetMap은 세계 지도를 자유롭게 편집할 수 있도록 만드는 협업 프로젝트입니다. [120]
- Afilias, .org, .info 등 도메인 등록부를 운영합니다. [121][122]
- 소니 온라인 멀티플레이어 온라인 게임. [123]
- BASF는 그들의 농업 비즈니스 포털을 위한 쇼핑 플랫폼입니다. [124]
- 레딧 소셜 뉴스 웹사이트입니다. [125]
- 스카이프 VoIP 애플리케이션, 중앙 비즈니스 데이터베이스. [126]
- Sun xVM, Sun의 가상화 및 데이터센터 자동화 제품군입니다. [127]
- MusicBrainz, 온라인 음악 백과사전을 열어보세요. [128]
- 국제우주정거장 – 궤도에서 원격 측정 데이터를 수집해 지상에 복제하는 역할입니다. [129]
- 마이 연감 소셜 네트워킹 사이트. [130]
- 인스타그램, 모바일 사진 공유 서비스입니다. [131]
- Disqus, 온라인 토론 및 댓글 서비스입니다. [132]
- 트립어드바이저, 주로 사용자 생성 콘텐츠의 여행 정보 웹사이트입니다. [133]
- 러시아 인터넷 기업 Yandex는 Yandex.Mail 서비스를 Oracle에서 Postgres로 전환했습니다. [134]
- Amazon Redshift는 ParAccel의 Postgres 수정을 기반으로 한 열형 온라인 분석 처리(OLAP) 시스템인 AWS의 일부입니다.
- 미국 해양대기청(NOAA) 국립기상청(NWS)의 인터랙티브 예보 준비 시스템(IFPS)은 NEXRAD 기상 레이더, 지상 및 수문학 시스템의 데이터를 통합하여 상세한 국지화 예보 모델을 구축하는 시스템입니다. [122][135]
- 영국의 국립 기상청인 기상청은 더 많은 오픈 소스 기술을 도입하기 위한 전략의 일환으로 오라클을 포스트그레SQL로 교체하기 시작했습니다. [135][136]
- WhitePages.com 오라클과 MySQL을 사용해왔지만, 핵심 디렉터리를 사내로 옮길 때 PostgreSQL로 전환했습니다. WhitePages.com 여러 출처의 대규모 데이터를 결합해야 하기 때문에, PostgreSQL이 높은 속도로 데이터를 로드하고 색인화할 수 있는 능력이 PostgreSQL을 선택한 결정의 핵심 요소였습니다. [122]
- FlightAware, 비행 추적 웹사이트입니다. [137]
- 그로퍼스, 온라인 식료품 배달 서비스입니다. [138]
- 가디언은 2018년에 MongoDB에서 PostgreSQL로 이전했습니다. [139]
- YugabyteDB는 PostgreSQL 쿼리 계층을 기본 SQL 모드로 구현합니다

- OpenAI는 ChatGPT를 포함해 주요 API 서비스의 일부로 PostgreSQL을 사용합니다. [140][141]

서비스 구현

몇몇 주목할 만한 공급업체들은 PostgreSQL을 서비스형 소프트웨어로 제공합니다:

- Heroku는 서비스 제공자로서 2007년 시작부터 PostgreSQL을 지원해 왔습니다. [142] 이들은 Heroku에서 개발한 오픈 소스 소프트웨어 WAL-E를 기반으로 한 완전한 데이터베이스 롤백(지정된 시점에 데이터베이스를 복원할 수 있는 기능)과 같은 부가가치 기능을 제공합니다[143]. [144]
- 2012년 1월, EnterpriseDB는 PostgreSQL과 자체 독점 Postgres Plus Advanced Server의 클라우드 버전을 출시했으며, 장애 조치, 복제, 부하 분산, 확장을 위한 자동 프로비저닝을 지원했습니다. 이 서비스는 아마존 웹 서비스에서 실행됩니다. [145] 2015년부터 Postgres Advanced Server는 ApsaraDB라는 PPAS라는 관계형 데이터베이스 서비스로 제공되고 있습니다. [146]
- VMware has offered vFabric Postgres (also termed vPostgres[147]) for private clouds on VMware vSphere since May 2012.[148] The company announced End of Availability (EOA) of the product in 2014.[149]
- In November 2013, Amazon Web Services announced the addition of PostgreSQL to their Relational Database Service offering.[150][151]
- In November 2016, Amazon Web Services announced the addition of PostgreSQL compatibility to their cloud-native Amazon Aurora managed database offering.[152]
- In May 2017, Microsoft Azure announced Azure Databases for PostgreSQL.[153]
- In May 2019, Alibaba Cloud announced PolarDB for PostgreSQL.[154]
- Jelastic Multicloud Platform as a Service has provided container-based PostgreSQL support since 2011. It also offers automated asynchronous master-slave replication of PostgreSQL.[155]
- In June 2019, IBM Cloud announced IBM Cloud Hyper Protect DBaaS for PostgreSQL.[156]
- In September 2020, Crunchy Data announced Crunchy Bridge.[157]
- In June 2022, Neon.tech announced Neon Serverless Postgres.[158]
- In December 2022, Google Cloud Platform announced general availability of AlloyDB as fully managed PostgreSQL cloud service.[159]
- In October 2023, Nile announced Nile Postgres Platform.[160]
- In April 2024, Google Cloud Platform announced general availability of AlloyDB Omni, a downloadable version of AlloyDB designed to run on any infrastructure, including on-premises, other clouds, or edge environments.[161]
- In September 2025, PlanetScale.com announced general availability of PlanetScale for Postgres, a managed, highly available version of Postgres.[162]

Release history

Release	First release	Latest minor version	Latest release	End of life[163]	Milestones
6.0	1997-01-29	— N/a	— N/a	— N/a	First formal release of PostgreSQL, unique indexes, pg_dumpall utility, ident authentication
6.1	1997-06-08	6.1.1	1997-07-22	— N/a	Multicolumn indexes, sequences, money data type, GEQO (GEnetic Query Optimizer)
6.2	1997-10-02	6.2.1	1997-10-17	— N/a	JDBC interface, triggers, server programming interface, constraints
6.3	1998-03-01	6.3.2	1998-04-07	2003-03-01	SQL-92 subselect ability, PL/pgTCL
6.4	1998-10-30	6.4.2	1998-12-20	2003-10-30	VIEWs (then only read-only) and RULEs, PL/pgSQL
6.5	1999-06-09	6.5.3	1999-10-13	2004-06-09	MVCC, temporary tables, more SQL statement support (CASE, INTERSECT, and EXCEPT)
7.0	2000-05-08	7.0.3	2000-11-11	2004-05-08	Foreign keys, SQL-92 syntax for joins
7.1	2001-04-13	7.1.3	2001-08-15	2006-04-13	Write-ahead log, outer joins
7.2	2002-02-04	7.2.8	2005-05-09	2007-02-04	PL/Python, OIDs no longer required, internationalization of messages
7.3	2002-11-27	7.3.21	2008-01-07	2007-11-27	Schema, table function, prepared query[164]

7.4	2003-11-17	7.4.30	2010-10-04	2010-10-01	Optimization on JOINs and data warehouse functions[165]
8.0	2005-01-19	8.0.26	2010-10-04	2010-10-01	Native server on Microsoft Windows, savepoints, tablespaces, point-in-time recovery[166]
8.1	2005-11-08	8.1.23	2010-12-16	2010-11-08	Performance optimization, two-phase commit, table partitioning, index bitmap scan, shared row locking, roles
8.2	2006-12-05	8.2.23	2011-12-05	2011-12-05	Performance optimization, online index builds, advisory locks, warm standby[167]
8.3	2008-02-04	8.3.23	2013-02-07	2013-02-07	Heap-only tuples, full text search,[168] SQL/XML, ENUM types, UUID types
8.4	2009-07-01	8.4.22	2014-07-24	2014-07-24	Window functions, column-level permissions, parallel database restore, per-database collation, common table expressions and recursive queries[169]
9.0	2010-09-20	9.0.23	2015-10-08	2015-10-08	Built-in binary streaming replication, hot standby, in-place upgrade ability, 64-bit Windows[170]
9.1	2011-09-12	9.1.24	2016-10-27	2016-10-27	Synchronous replication, per-column collations, unlogged tables, serializable snapshot isolation, writeable common table expressions, SELinux integration, extensions, foreign tables[171]
9.2	2012-09-10[172]	9.2.24	2017-11-09	2017-11-09	Cascading streaming replication, index-only scans, native JSON support, improved lock management, range types, pg_receivexlog tool, space-partitioned GiST indexes

9.3	2013-09-09	9.3.25	2018-11-08	2018-11-08	Custom background workers, data checksums, dedicated JSON operators, LATERAL JOIN, faster pg_dump, new pg_isready server monitoring tool, trigger features, view features, writeable foreign tables, materialized views, replication improvements
9.4	2014-12-18	9.4.26	2020-02-13	2020-02-13	JSONB 데이터 타입, 설정 값 변경을 위한 ALTER SYSTEM 문, 읽기 차단 없이 물질화된 뷰를 새로고침하는 기능, 백그라운드 워커 프로세스의 동적 등록/시작/중지, 논리 디코딩 API, GiN 인덱스 개선, 리눅스 대형 페이지 지원, pg_prewarm를 통한 데이터베이스 캐시 재로딩, 문서 스타일 데이터의 열 타입으로 H스토어를 다시 도입했습니다. [173]
9.5	2016-01-07	9.5.25	2021-02-11	2021-02-11	UPSERT, 행 수준 보안, TABLESAMPLE, CUBE/ROLLUP, 집합 그룹화, 그리고 새로운 BRIN 인덱스[174]
9.6	2016-09-29	9.6.24	2021-11-11	2021-11-11	병렬 쿼리 지원, 정렬/조인 푸시다운을 통한 PostgreSQL 외래퍼 래퍼(FDW) 개선, 다중 동기 대기, 대형 테이블의 빠른 진공 추출
10	2017-10-05	10.23	2022-11-10	2022-11-10	논리적 복제,[175] 선언적 테이블 분할, 향상된 쿼리 병렬성
11	2018-10-18	11.22	2023-11-09	2023-11-09	파티셔닝의 견고성과 성능 향상, 저장 프로시저에서 트랜잭션 지원, 쿼리 병렬성 향상, 표현식에 대한 적시(JIT) 컴파일[176][177]
12	2019-10-03	12.22	2024-11-21	2024-11-21	쿼리 성능 및 공간 활용 개선; SQL/JSON 경로식 지원; 생성된 열; 국제화 및 인증 개선; 새로운 플러그인 테이블 스토리지 인터페이스. [178]

13	2020-09-24	13.23	2025-11-13	2025-11-13	B-트리 인덱스 항목의 중복 제거로 인한 공간 절감과 성능 향상, 집계 또는 분할 테이블을 사용하는 쿼리의 성능 향상, 확장 통계 사용 시 더 나은 쿼리 계획, 인덱스의 병렬 진공 처리, 점분 정렬[179][180]
14	2021-09-30	14.23	2026-05-14	2026-11-12	일반적인 테이블 표현식에 대한 SQL 표준 SEARCH와 CYCLE 절을 추가하여 DISTINCT를 GROUP BY에 추가할 수 있게 했습니다[181][182]
15	2022-10-13	15.18	2026-05-14	2027-11-11	SQL 표준 MERGE 문이 구현됨. PL/Python은 현재 Python 3만 지원하며, 단종된 Python 2가 아닌 Python 3를 의미합니다. `plpythonu`
16	2023-09-14	16.14	2026-05-14	2028-11-09	논리 복제의 개선, pg_stat_io 뷰 (I/O 메트릭에 한)[183]
17	2024-09-26	17.10	2026-05-14	2029-11-08	진공 프로세스, I/O 계층, 쿼리 실행의 성능 향상, JSON 기능 확장, MERGE 기능 증가, COPY 개선; 고가용성과 업그레이드를 위한 논리적 복제를 강화하며, 보안, 운영, 모니터링, 분석 개선을 지원합니다. [184]
18	2025-09-25	18.4	2026-05-14	2030-11-14	새로운 I/O 서브시스템과 비동기식 I/O 향상. [185]

전설:

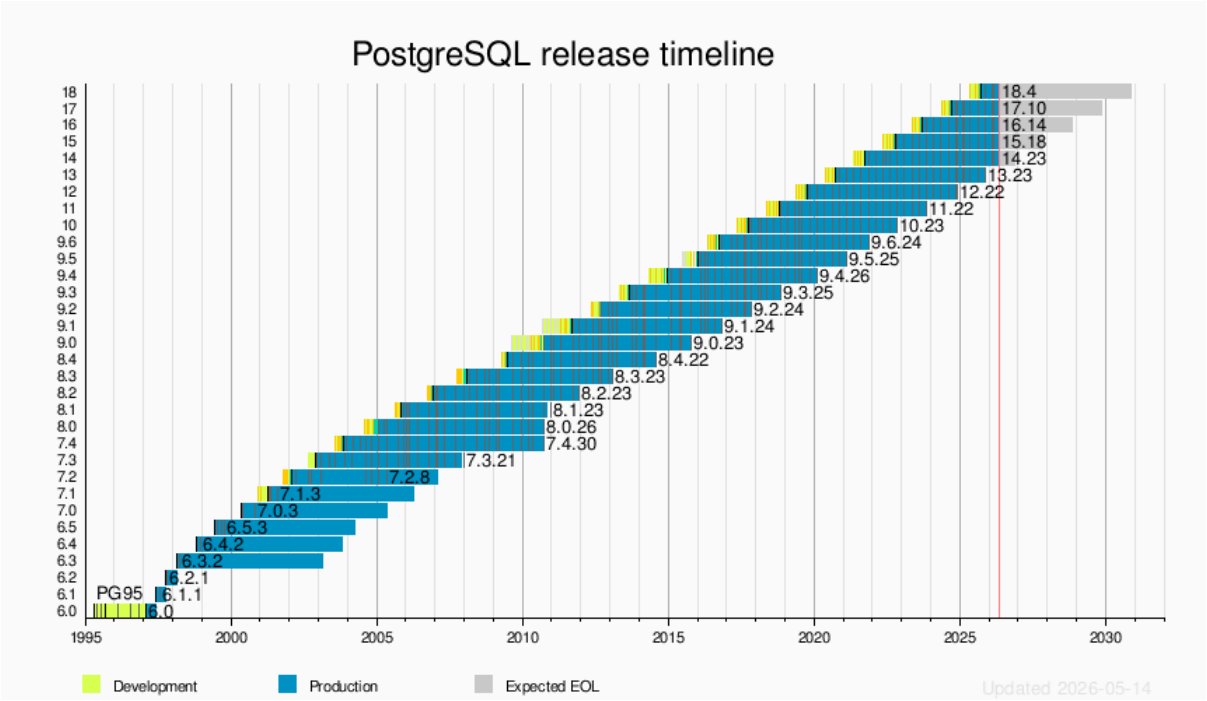
지원 없음

지원

최신 버전

미리보기 버전

미래 버전



생태계와 파생 생물

포스트그레SQL의 관대한 오픈 소스 라이선스와 확장 가능한 아키텍처 덕분에 광범위한 생태계가 형성되었습니다. 여기에는 전용 지원과 호스팅을 제공하는 수많은 회사뿐만 아니라, 특정 워크로드에 맞게 PostgreSQL을 적응시키는 여러 포크 및 파생 데이터베이스가 포함됩니다. 주요 파생상품은 다음과 같습니다:

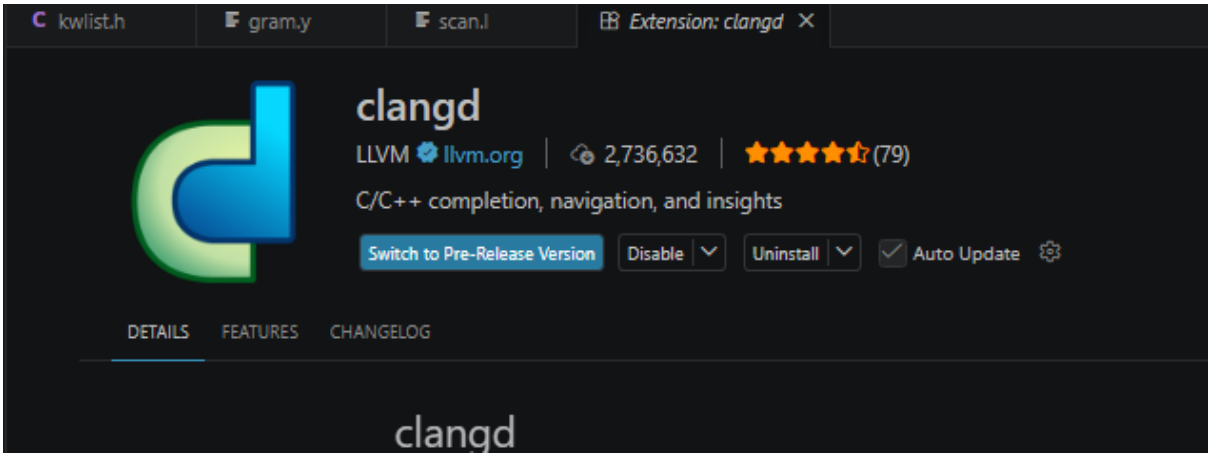
- 그린플럼 데이터베이스: 대규모 분석용으로 설계된 구버전 PostgreSQL 기반의 대규모 병렬 처리(MPP) 데이터 웨어하우스입니다. [186]
- TimescaleDB: PostgreSQL 확장 기능으로 제공되는 시계열 데이터베이스로, 시계열 데이터의 빠른 인제스트와 복잡한 쿼리를 처리하도록 최적화되어 있습니다. [187]
- Amazon Aurora: Amazon Web Services에서 제공하는 클라우드 네이티브 관계형 데이터베이스로, PostgreSQL 호환 에디션을 제공합니다. [188]
- Neon: 저장소와 컴퓨트를 분리하여 데이터베이스 분기와 같은 현대적인 개발 기능을 제공하는 오픈소스 서버리스 PostgreSQL의 구현체입니다. [189]
- AlloyDB: 컴퓨트와 스토리지를 분리하는 완전 관리형 PostgreSQL 호환 구글 클라우드 데이터베이스로, 하이브리드 워크로드와 통합 AI 기능을 위해 설계되었습니다. [190]
- PlanetScale Postgres: 데이터베이스 분기와 데이터베이스 트래픽 제어™를 지원하는 완전 관리되고 고가용성의 Postgres 인스턴스입니다. [162][191]

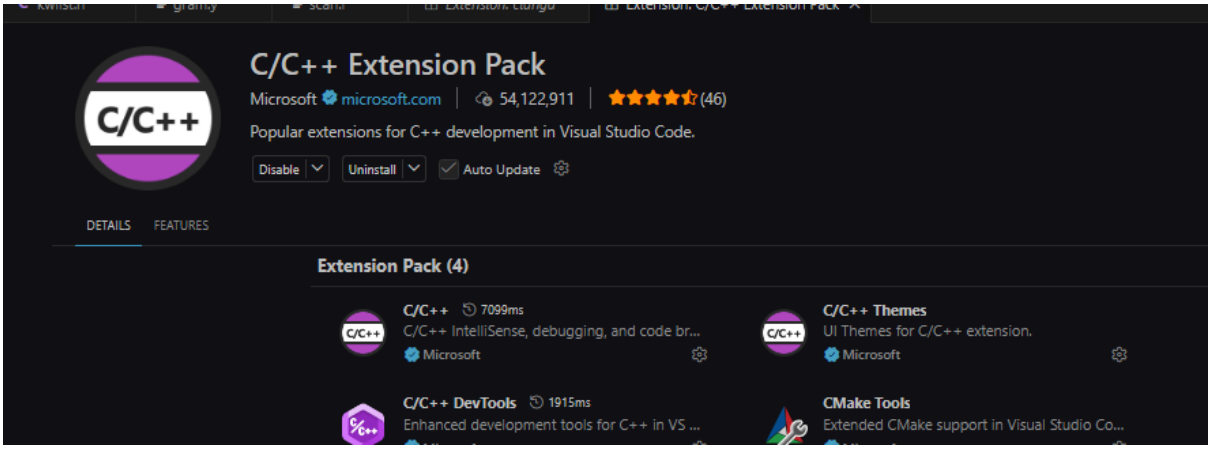
PostgreSQL - GitHub

PostgreSQL 16.15 버전 소스코드

[postgres/postgres: Mirror of the official PostgreSQL GIT repository. Note that this is just a *mirror* - we don't work with pull requests on github. To contribute, please see https://wiki.postgresql.org/wiki/Submitting_a_Patch]

- vscode 설치환경 설정
- extension 설치

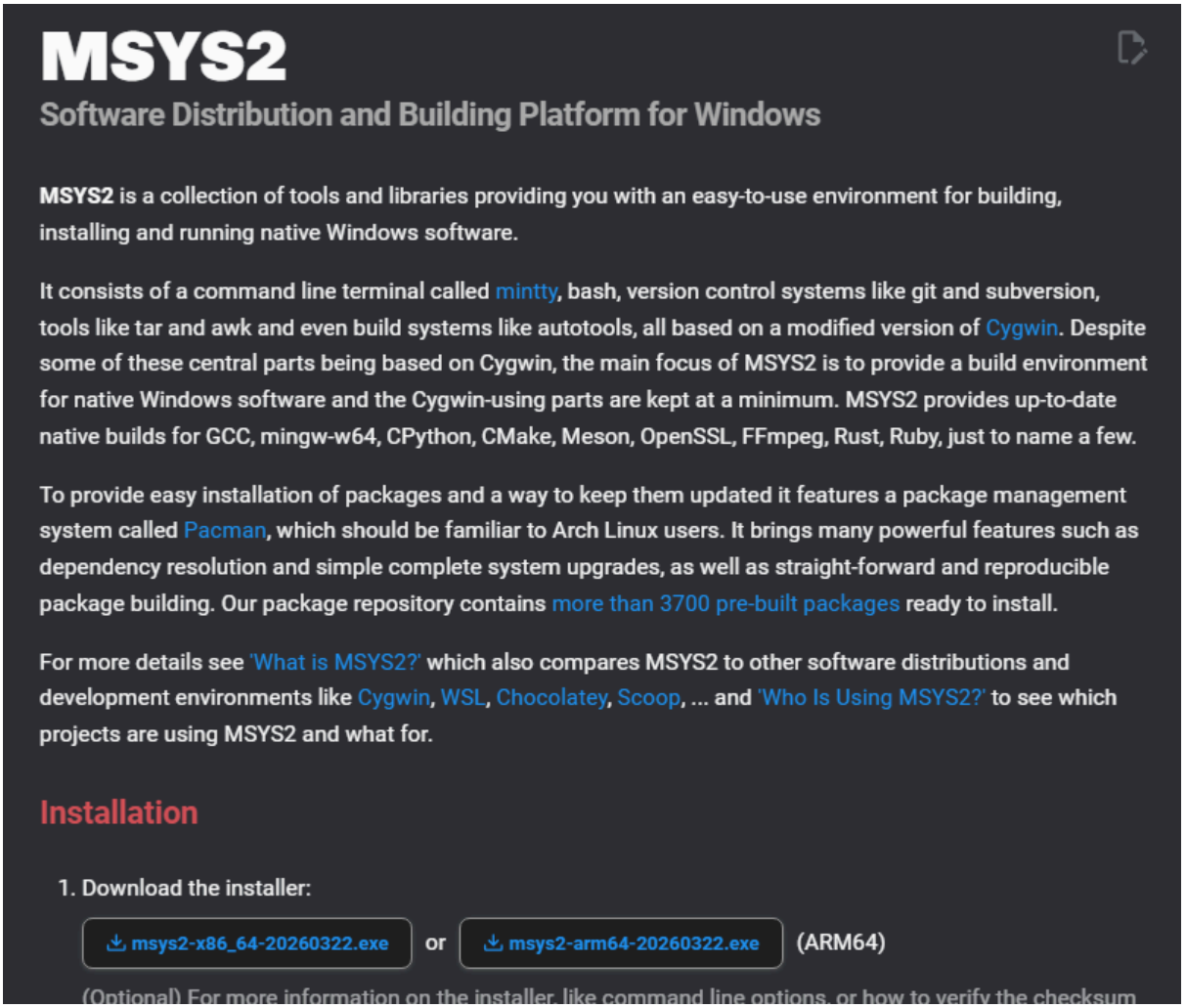




- LLVM설치

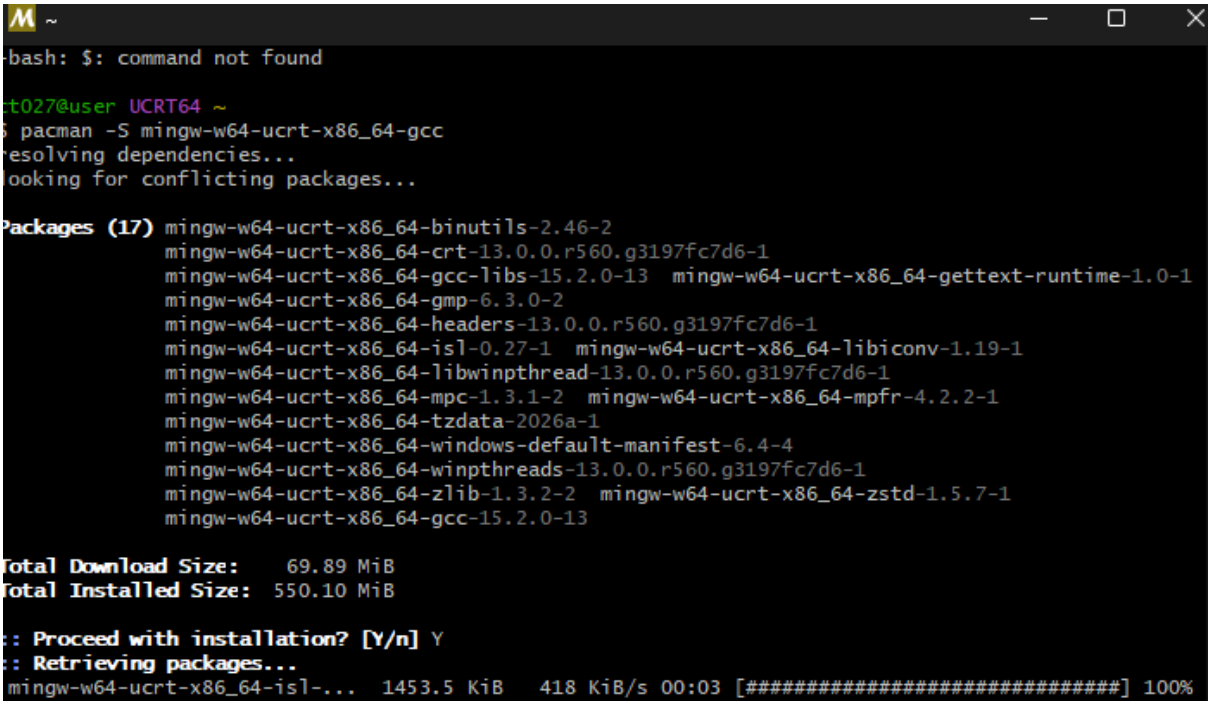


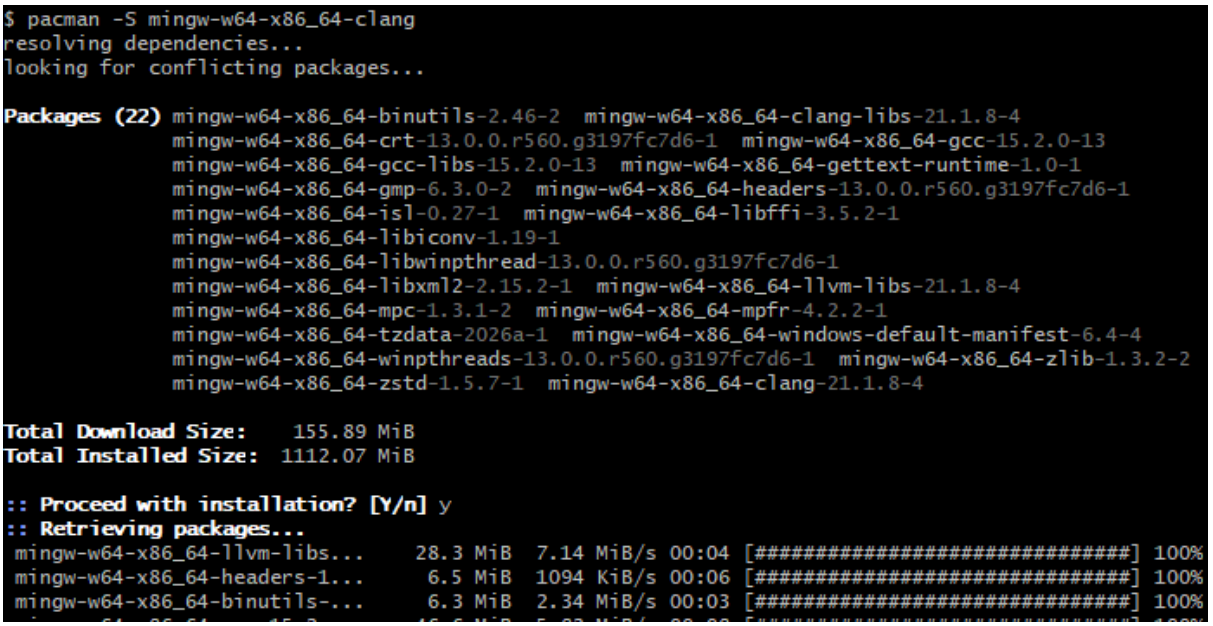
- MSYS2 설치

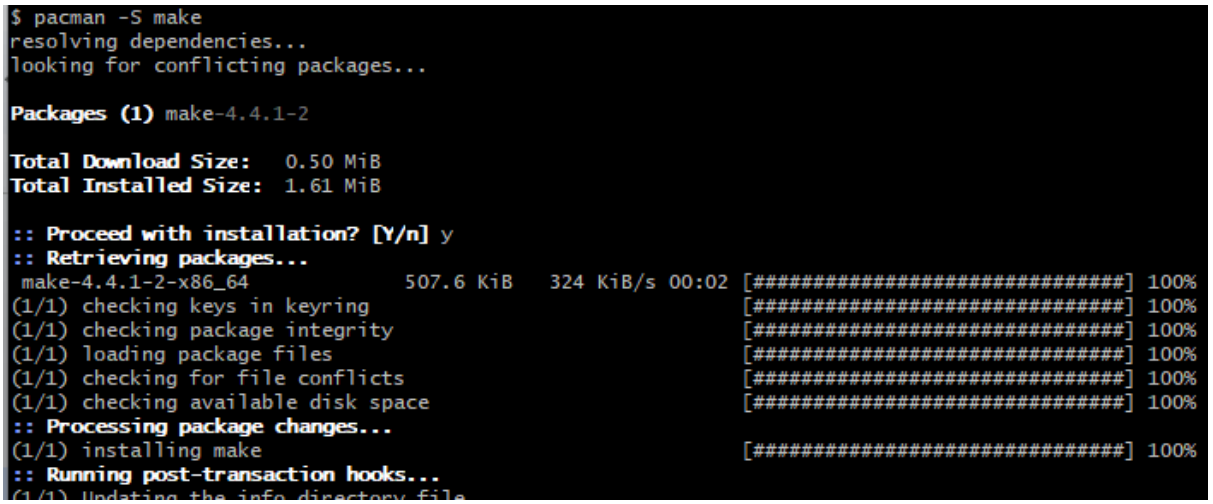


- gcc 설치

pacman -S mingw-w64-x86_64-gcc
pacman -S mingw-w64-x86_64-clang
pacman -S make


~
-bash: $: command not found
t027@user UCRT64 ~
$ pacman -S mingw-w64-ucrt-x86_64-gcc
resolving dependencies...
looking for conflicting packages...
Packages (17) mingw-w64-ucrt-x86_64-binutils-2.46-2
mingw-w64-ucrt-x86_64-crt-13.0.0.r560.g3197fc7d6-1
mingw-w64-ucrt-x86_64-gcc-libs-15.2.0-13 mingw-w64-ucrt-x86_64-gettext-runtime-1.0-1
mingw-w64-ucrt-x86_64-gmp-6.3.0-2
mingw-w64-ucrt-x86_64-headers-13.0.0.r560.g3197fc7d6-1
mingw-w64-ucrt-x86_64-isl-0.27-1 mingw-w64-ucrt-x86_64-libiconv-1.19-1
mingw-w64-ucrt-x86_64-libwinpthread-13.0.0.r560.g3197fc7d6-1
mingw-w64-ucrt-x86_64-mpc-1.3.1-2 mingw-w64-ucrt-x86_64-mpfr-4.2.2-1
mingw-w64-ucrt-x86_64-tzdata-2026a-1
mingw-w64-ucrt-x86_64-windows-default-manifest-6.4-4
mingw-w64-ucrt-x86_64-winpthreads-13.0.0.r560.g3197fc7d6-1
mingw-w64-ucrt-x86_64-zlib-1.3.2-2 mingw-w64-ucrt-x86_64-zstd-1.5.7-1
mingw-w64-ucrt-x86_64-gcc-15.2.0-13
Total Download Size: 69.89 MiB
Total Installed Size: 550.10 MiB
:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
mingw-w64-ucrt-x86_64-isl-... 1453.5 KiB 418 KiB/s 00:03 [############################] 100%


$ pacman -S mingw-w64-x86_64-clang
resolving dependencies...
looking for conflicting packages...
Packages (22) mingw-w64-x86_64-binutils-2.46-2 mingw-w64-x86_64-clang-libs-21.1.8-4
mingw-w64-x86_64-crt-13.0.0.r560.g3197fc7d6-1 mingw-w64-x86_64-gcc-15.2.0-13
mingw-w64-x86_64-gcc-libs-15.2.0-13 mingw-w64-x86_64-gettext-runtime-1.0-1
mingw-w64-x86_64-gmp-6.3.0-2 mingw-w64-x86_64-headers-13.0.0.r560.g3197fc7d6-1
mingw-w64-x86_64-isl-0.27-1 mingw-w64-x86_64-libffi-3.5.2-1
mingw-w64-x86_64-libiconv-1.19-1
mingw-w64-x86_64-libwinpthread-13.0.0.r560.g3197fc7d6-1
mingw-w64-x86_64-libxml2-2.15.2-1 mingw-w64-x86_64-llvm-libs-21.1.8-4
mingw-w64-x86_64-mpc-1.3.1-2 mingw-w64-x86_64-mpfr-4.2.2-1
mingw-w64-x86_64-tzdata-2026a-1 mingw-w64-x86_64-windows-default-manifest-6.4-4
mingw-w64-x86_64-winpthreads-13.0.0.r560.g3197fc7d6-1 mingw-w64-x86_64-zlib-1.3.2-2
mingw-w64-x86_64-zstd-1.5.7-1 mingw-w64-x86_64-clang-21.1.8-4
Total Download Size: 155.89 MiB
Total Installed Size: 1112.07 MiB
:: Proceed with installation? [Y/n] y
:: Retrieving packages...
mingw-w64-x86_64-llvm-libs... 28.3 MiB 7.14 MiB/s 00:04 [############################] 100%
mingw-w64-x86_64-headers-1... 6.5 MiB 1094 KiB/s 00:06 [############################] 100%
mingw-w64-x86_64-binutils-... 6.3 MiB 2.34 MiB/s 00:03 [############################] 100%

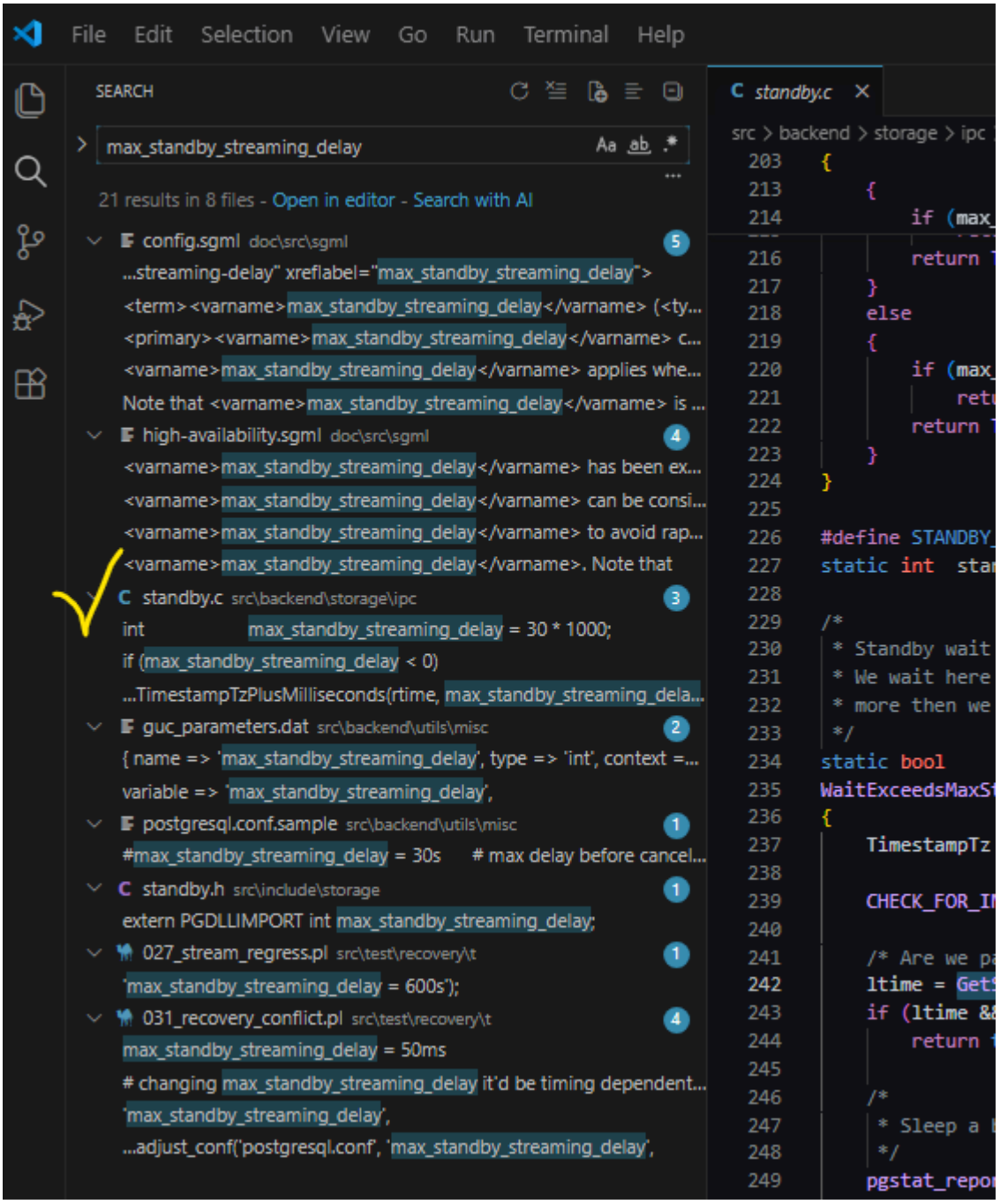

$ pacman -S make
resolving dependencies...
looking for conflicting packages...
Packages (1) make-4.4.1-2
Total Download Size: 0.50 MiB
Total Installed Size: 1.61 MiB
:: Proceed with installation? [Y/n] y
:: Retrieving packages...
make-4.4.1-2-x86_64 507.6 KiB 324 KiB/s 00:02 [############################] 100%
(1/1) checking keys in keyring [############################] 100%
(1/1) checking package integrity [############################] 100%
(1/1) loading package files [############################] 100%
(1/1) checking for file conflicts [############################] 100%
(1/1) checking available disk space [############################] 100%
:: Processing package changes...
(1/1) installing make [############################] 100%
:: Running post-transaction hooks...

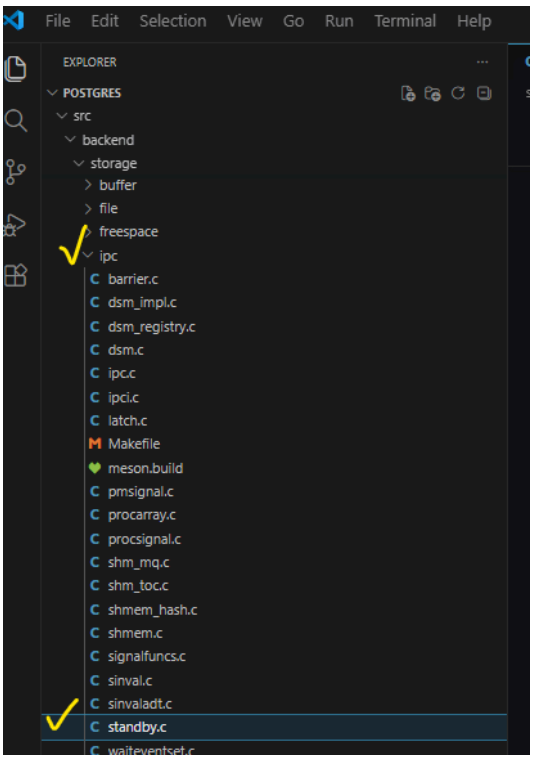
- configure

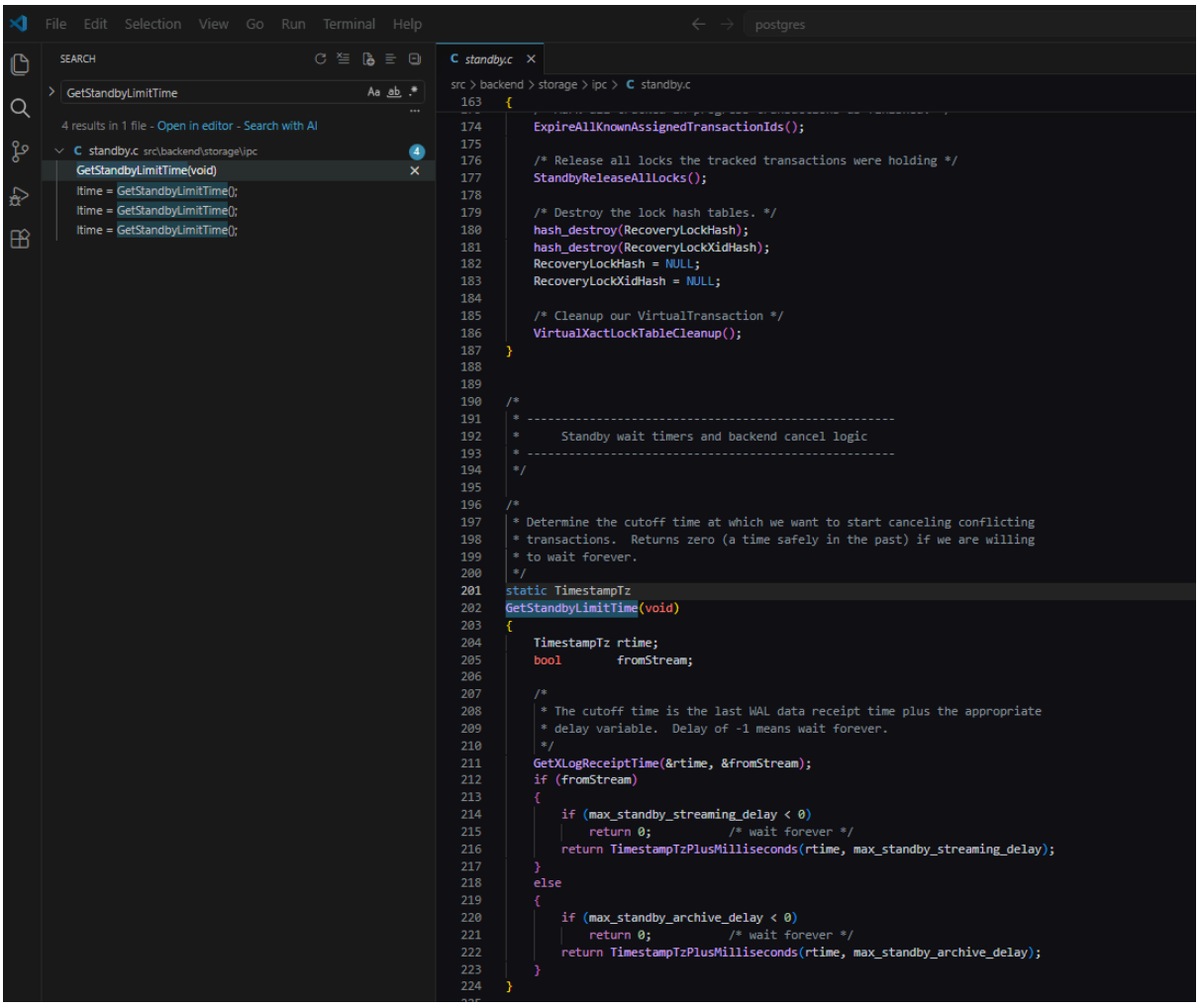
```
ct027@user UCRT64 /d/code/postgres
$ ./configure
configure: loading site script /etc/config.site
checking build system type... x86_64-w64-mingw32
checking host system type... x86_64-w64-mingw32
checking which template to use... win32
checking whether NLS is wanted... no
checking for default port number... 5432
checking for block size... 8kB
checking for segment size... 1GB
checking for WAL block size... 8kB
checking for gcc... gcc
checking whether the C compiler works...
```

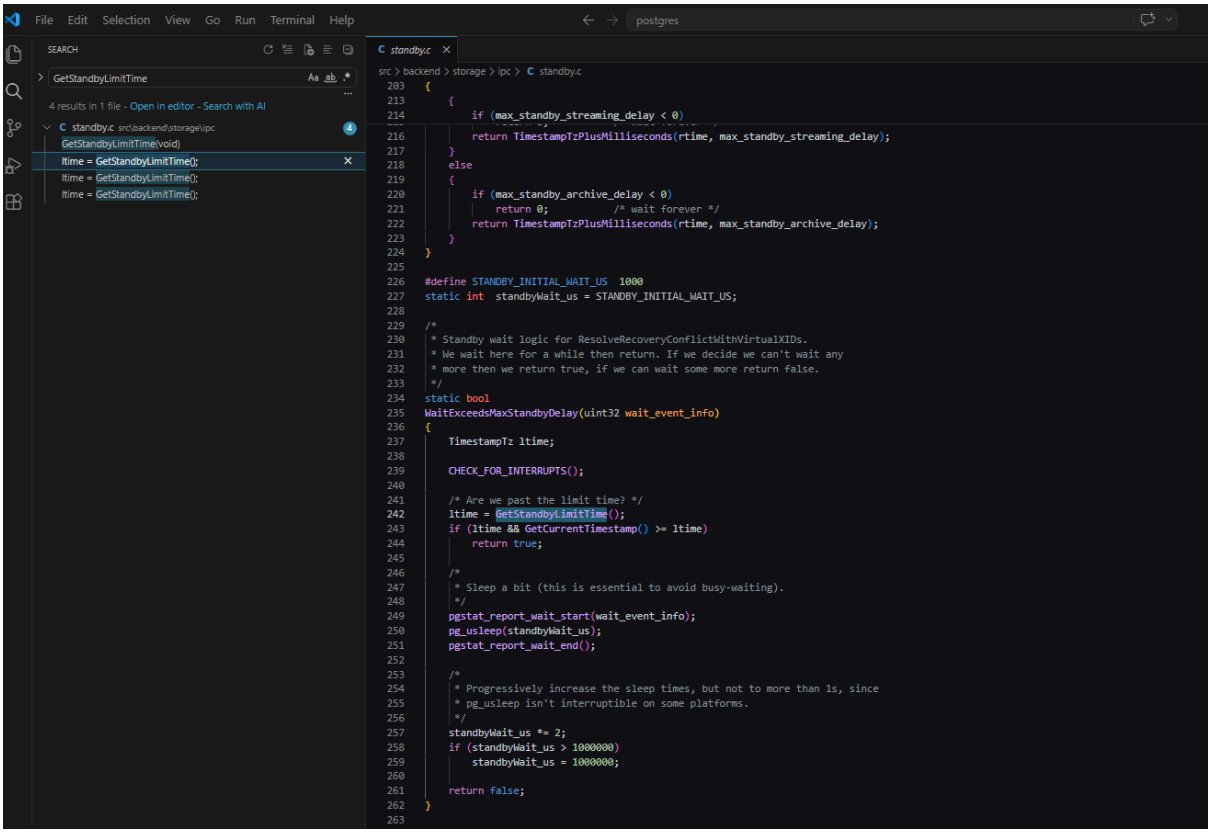
예제1> `max_standby_streaming_delay`

PostgreSQL 리플리카스(standby)에서 WAL 적용(replay)과 조회 쿼리 충돌 시 얼마나 기다릴지 정하는 설정 단위: 밀리초(ms)




File Edit Selection View Go Run Terminal Help
EXPLORER
POSTGRES
src
backend
storage
buffer
file
freespace
ipc
barrier.c
dsm_impl.c
dsm_registry.c
dsm.c
ipc.c
ipci.c
latch.c
Makefile
meson.build
pmsignal.c
procarray.c
procsignal.c
shm_mq.c
shm_toc.c
shmem_hash.c
shmem.c
signalfuncs.c
sinval.c
sinvaladt.c
standby.c


File Edit Selection View Go Run Terminal Help
postgres
SEARCH
GetStandbyLimitTime
4 results in 1 file - Open in editor - Search with AI
standby.c src\backend\storage\ipc
GetStandbyLimitTime(void)
ltime = GetStandbyLimitTime();
ltime = GetStandbyLimitTime();
ltime = GetStandbyLimitTime();
standby.c
src > backend > storage > ipc > standby.c
{
ExpireAllKnownAssignedTransactionIds();
/* Release all locks the tracked transactions were holding */
StandbyReleaseAllLocks();
/* Destroy the lock hash tables. */
hash_destroy(RecoveryLockHash);
hash_destroy(RecoveryLockXidHash);
RecoveryLockHash = NULL;
RecoveryLockXidHash = NULL;
/* Cleanup our VirtualTransaction */
VirtualXactLockTableCleanup();
}
/*
* ---
* Standby wait timers and backend cancel logic
* ---
*/
/*
* Determine the cutoff time at which we want to start canceling conflicting
* transactions. Returns zero (a time safely in the past) if we are willing
* to wait forever.
*/
static TimestampTz
GetStandbyLimitTime(void)
{
TimestampTz rtime;
bool fromStream;
/*
* The cutoff time is the last WAL data receipt time plus the appropriate
* delay variable. Delay of -1 means wait forever.
*/
GetXLogReceiptTime(&rtime, &fromStream);
if (fromStream)
{
if (max_standby_streaming_delay < 0)
return 0; /* wait forever */
return TimestampTzPlusMilliseconds(rtime, max_standby_streaming_delay);
}
else
{
if (max_standby_archive_delay < 0)
return 0; /* wait forever */
return TimestampTzPlusMilliseconds(rtime, max_standby_archive_delay);
}
}



예제2> SQL Lex Parser
kwlist.h에 등록된 SELECT 문장의 키워드를 스캔하고 scan.l 렉스파서에 전달하면 gram.y bison/yacc 기반 parser가 문법해석하고 parser.c를 출력하는 형식.

Keyword 목록

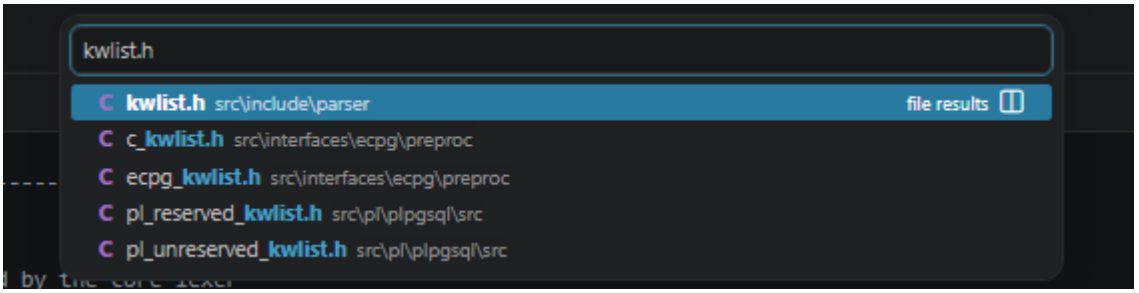
```
src/include/parser/kwlist.h
```

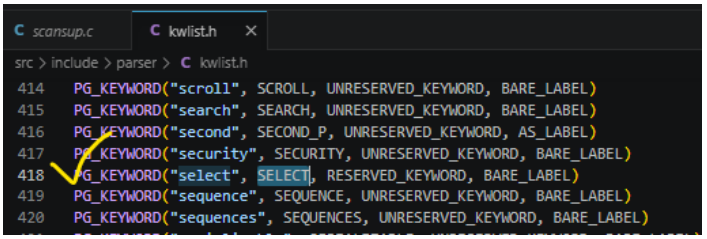
reserved / unreserved keyword 관리.

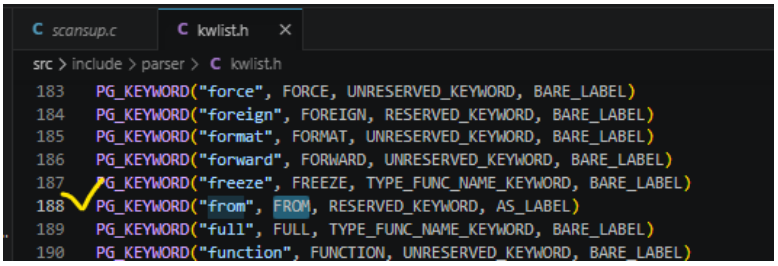
Parser 흐름

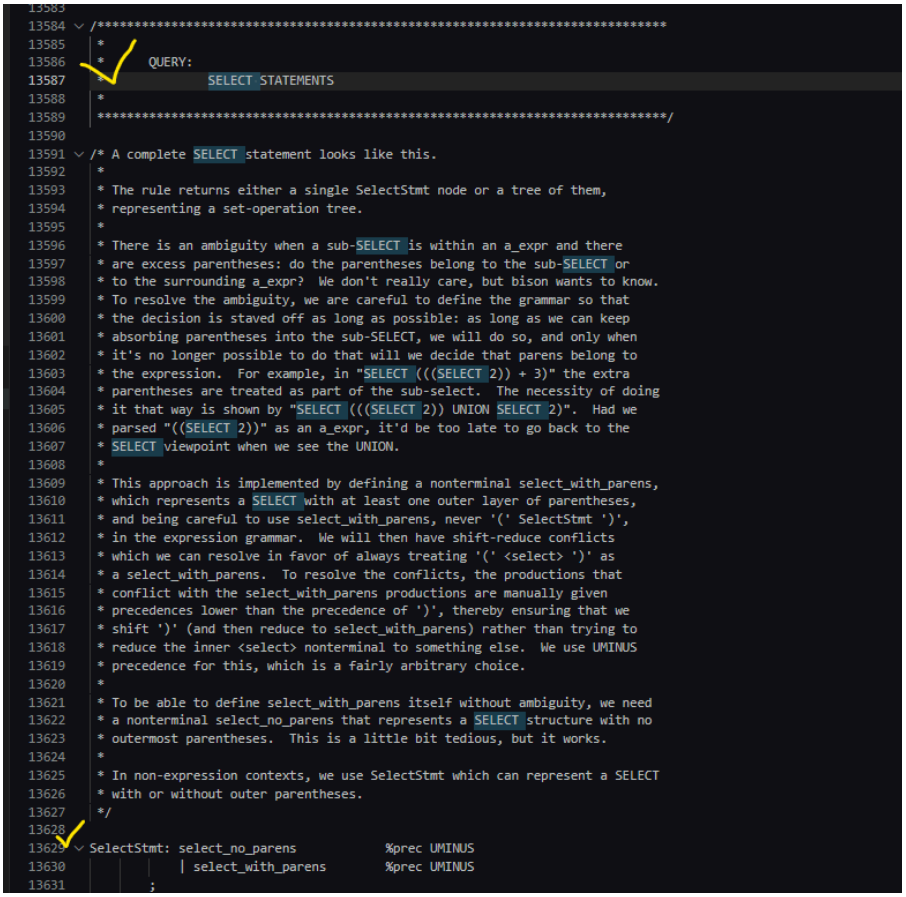
대략:

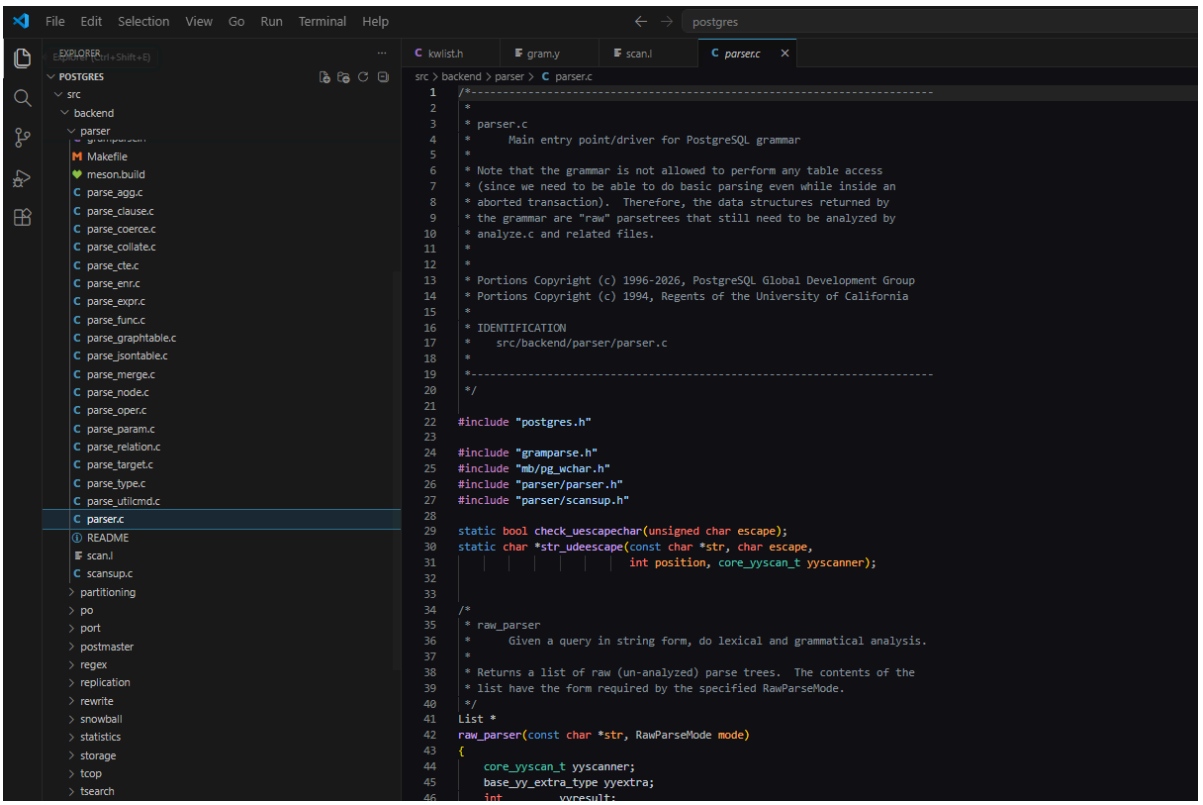
```
SQL text
  ↓
scan.l (lexer)
  ↓ token
gram.y (parser)
  ↓ parse tree
parser.c
  ↓
analyzer/rewrite/planner
```


kwlist.h
kwlist.h src\include\parser
file results
c_kwlist.h src\interfaces\ecpg\preproc
ecpg_kwlist.h src\interfaces\ecpg\preproc
pl_reserved_kwlist.h src\pl\plpgsql\src
pl_unreserved_kwlist.h src\pl\plpgsql\src


scansup.c
kwlist.h
src > include > parser > kwlist.h
414 PG_KEYWORD("scroll", SCROLL, UNRESERVED_KEYWORD, BARE_LABEL)
415 PG_KEYWORD("search", SEARCH, UNRESERVED_KEYWORD, BARE_LABEL)
416 PG_KEYWORD("second", SECOND_P, UNRESERVED_KEYWORD, AS_LABEL)
417 PG_KEYWORD("security", SECURITY, UNRESERVED_KEYWORD, BARE_LABEL)
418 PG_KEYWORD("select", SELECT, RESERVED_KEYWORD, BARE_LABEL)
419 PG_KEYWORD("sequence", SEQUENCE, UNRESERVED_KEYWORD, BARE_LABEL)
420 PG_KEYWORD("sequences", SEQUENCES, UNRESERVED_KEYWORD, BARE_LABEL)


scansup.c
kwlist.h
src > include > parser > kwlist.h
183 PG_KEYWORD("force", FORCE, UNRESERVED_KEYWORD, BARE_LABEL)
184 PG_KEYWORD("foreign", FOREIGN, RESERVED_KEYWORD, BARE_LABEL)
185 PG_KEYWORD("format", FORMAT, UNRESERVED_KEYWORD, BARE_LABEL)
186 PG_KEYWORD("forward", FORWARD, UNRESERVED_KEYWORD, BARE_LABEL)
187 PG_KEYWORD("freeze", FREEZE, TYPE_FUNC_NAME_KEYWORD, BARE_LABEL)
188 PG_KEYWORD("from", FROM, RESERVED_KEYWORD, AS_LABEL)
189 PG_KEYWORD("full", FULL, TYPE_FUNC_NAME_KEYWORD, BARE_LABEL)
190 PG_KEYWORD("function", FUNCTION, UNRESERVED_KEYWORD, BARE_LABEL)


13584 /***
13585 *
13586 * QUERY:
13587 * SELECT STATEMENTS
13588 *
13589 ***/
13590
13591 /* A complete SELECT statement looks like this.
13592 *
13593 * The rule returns either a single SelectStmt node or a tree of them,
13594 * representing a set-operation tree.
13595 *
13596 * There is an ambiguity when a sub-SELECT is within an a_expr and there
13597 * are excess parentheses: do the parentheses belong to the sub-SELECT or
13598 * to the surrounding a_expr? We don't really care, but bison wants to know.
13599 * To resolve the ambiguity, we are careful to define the grammar so that
13600 * the decision is staved off as long as possible: as long as we can keep
13601 * absorbing parentheses into the sub-SELECT, we will do so, and only when
13602 * it's no longer possible to do that will we decide that parens belong to
13603 * the expression. For example, in "SELECT (((SELECT 2)) + 3)" the extra
13604 * parentheses are treated as part of the sub-select. The necessity of doing
13605 * it that way is shown by "SELECT (((SELECT 2)) UNION SELECT 2)". Had we
13606 * parsed "((SELECT 2))" as an a_expr, it'd be too late to go back to the
13607 * SELECT viewpoint when we see the UNION.
13608 *
13609 * This approach is implemented by defining a nonterminal select_with_parens,
13610 * which represents a SELECT with at least one outer layer of parentheses,
13611 * and being careful to use select_with_parens, never '(' SelectStmt ')',
13612 * in the expression grammar. We will then have shift-reduce conflicts
13613 * which we can resolve in favor of always treating '(' <select> ')' as
13614 * a select_with_parens. To resolve the conflicts, the productions that
13615 * conflict with the select_with_parens productions are manually given
13616 * precedences lower than the precedence of ')', thereby ensuring that we
13617 * shift ')' (and then reduce to select_with_parens) rather than trying to
13618 * reduce the inner <select> nonterminal to something else. We use UMINUS
13619 * precedence for this, which is a fairly arbitrary choice.
13620 *
13621 * To be able to define select_with_parens itself without ambiguity, we need
13622 * a nonterminal select_no_parens that represents a SELECT structure with no
13623 * outermost parentheses. This is a little bit tedious, but it works.
13624 *
13625 * In non-expression contexts, we use SelectStmt which can represent a SELECT
13626 * with or without outer parentheses.
13627 */
13628
13629 SelectStmt: select_no_parens %prec UMINUS
13630 | select_with_parens %prec UMINUS
13631 ;


File Edit Selection View Go Run Terminal Help
postgres
EXPLORER
POSTGRES
src
backend
parser
Makefile
meson.build
parse_agg.c
parse_clause.c
parse_coerce.c
parse_collate.c
parse_cte.c
parse_enr.c
parse_expr.c
parse_func.c
parse_graphtable.c
parse_jsontable.c
parse_merge.c
parse_node.c
parse_oper.c
parse_param.c
parse_relation.c
parse_target.c
parse_type.c
parse_utilcmd.c
parser.c
README
scan.l
scansup.c
partitioning
po
port
postmaster
regex
replication
rewrite
snowball
statistics
storage
tcop
tsearch
kwlist.h
gram.y
scan.l
parser.c
src > backend > parser > parser.c
/*---
*
* parser.c
*   Main entry point/driver for PostgreSQL grammar
*
* Note that the grammar is not allowed to perform any table access
* (since we need to be able to do basic parsing even while inside an
* aborted transaction). Therefore, the data structures returned by
* the grammar are "raw" parsetrees that still need to be analyzed by
* analyze.c and related files.
*
*
* Portions Copyright (c) 1996-2026, PostgreSQL Global Development Group
* Portions Copyright (c) 1994, Regents of the University of California
*
* IDENTIFICATION
*   src/backend/parser/parser.c
*
*---
*/
#include "postgres.h"
#include "gramparse.h"
#include "mb/pg_wchar.h"
#include "parser/parser.h"
#include "parser/scansup.h"
static bool check_uescapechar(unsigned char escape);
static char *str_udeescape(const char *str, char escape,
int position, core_yyscan_t yyscanner);
/*
* raw_parser
*   Given a query in string form, do lexical and grammatical analysis.
*
* Returns a list of raw (un-analyzed) parse trees. The contents of the
* list have the form required by the specified RawParseMode.
*/
List *
raw_parser(const char *str, RawParseMode mode)
{
core_yyscan_t yyscanner;
base_yy_extra_type yyextra;
int yyresult;